



Large language models for cyberattack defense: a critical survey

Lamia Chaari Fourati¹ · Mirna Awad² · Mariem Ben Ali^{1,2} · Wael Jaafar²

Received: 10 November 2025 / Revised: 18 February 2026 / Accepted: 5 March 2026
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2026

Abstract

Large Language Models (LLMs) are increasingly explored as defender-side intelligence components that can interpret heterogeneous security artifacts and support decision-making across the cyber defense lifecycle. In network and application security, recent studies investigate LLMs not only as semantic reasoning engines for unstructured inputs (e.g., alerts, logs, threat intelligence narratives, phishing content, and code artifacts), but also as modules integrated into IDS/SOC pipelines for detection, triage, explanation, and as retrieval- and tool-augmented agents that assist mitigation and response under controlled execution. This survey provides an analysis-driven review of LLM-based approaches for cyberattack detection and mitigation across major attack families, including DoS/DDoS, phishing, malware, SQL injection, cross-site scripting (XSS), eavesdropping, and man-in-the-middle (MITM). Beyond cataloging the literature, we introduce a unified taxonomy that separates detection from mitigation objectives and systematically compare representative solutions in terms of modeling strategy (prompting, fine-tuning, retrieval-augmented generation, and agentic tool use), data modality, and evaluation setting. We further emphasize feasibility by discussing operational constraints such as latency and throughput, integration with SIEM/SOAR/EDR platforms, compute and maintenance costs, and generalization under domain shift. Finally, we examine LLM-specific risks that affect real-world adoption, including prompt injection, poisoned retrieval context, hallucinated recommendations, and tool/agent misuse, and summarize assurance mechanisms and open research challenges toward reliable and deployable LLM-enabled cyber defense.

Keywords Large language models · LLMs · Network security · Cyber attack detection · Intrusion detection

Abbreviations

AI	Artificial intelligence
ANN	Artificial neural network
API	Application programming interface
APT	Advanced persistent threat

✉ Lamia Chaari Fourati
lamia.chaari@isims.usf.tn

¹ Digital Research Center of Sfax (CRNS), Laboratory of Signals, Systems, Artificial Intelligence, NetworkS (SM@RTS), Sfax University, Sfax, Tunisia

² École de Technologie Supérieure (ÉTS), Montreal, Canada

AUC	Area under the curve
CNN	Convolutional neural network
CSRF	Cross-site request forgery
CTI	Cyber threat intelligence
CVE	Common vulnerabilities and exposures
DDoS	Distributed denial of service
DL	Deep learning
DLP	Data loss prevention
DoS	Denial of service
DOM	Document object model
EDR	Endpoint detection and response
FM	Foundation model
GAN	Generative adversarial network
GDPR	General data protection regulation
GenAI	Generative artificial intelligence
GNN	Graph neural network
HTML	HyperText markup language
IDS	Intrusion detection system
IIoT	Industrial Internet of Things
IOC	Indicator of compromise
IoMT	Internet of medical things
IoT	Internet of Things
IPS	Intrusion prevention system
LLM	Large language model
LSTM	Long short-term memory
MITM	Man-in-the-middle
ML	Machine learning
MLLM	Multimodal large language model
NLP	Natural language processing
OWASP	Open Worldwide Application Security Project
PCAP	Packet capture
PI	Prompt injection
QBER	Quantum bit error rate
QKD	Quantum key distribution
QKDP	Quantum key distribution protocol
RAG	Retrieval-augmented generation
RF	Random forest
RNN	Recurrent neural network
SIEM	Security information and event management
SOAR	Security orchestration, automation, and response
SOC	Security operations center
SQLi	SQL injection
SVM	Support vector machine
TTP	Tactics, techniques, and procedures
URL	Uniform resource locator
WAF	Web application firewall
XSS	Cross-site scripting
XSRF	Cross-site request forgery

1 Introduction

1.1 General context

The rapidly evolving digital landscape has made network security a critical focus in safeguarding the integrity, confidentiality, and availability of critical infrastructures [1]. Cyberattacks, including distributed denial of service (DDoS), ransomware, botnet traffic, and man-in-the-middle (MITM) attacks, are growing in complexity, pushing researchers and engineers to develop increasingly robust and intelligent security solutions.[2, 3].

In response, recent research investigates large language models (LLMs) [4] as defender-side intelligence components [5] that can interpret and reason over heterogeneous security artifacts [6], augment intrusion detection [7, 8] and analysis pipelines, and support mitigation and response through retrieval-and-tool-augmented workflows [9–11].

LLMs are increasingly explored to process unstructured and semi-structured security data (e.g., alerts, logs, incident reports, threat intelligence narratives, phishing content, and code artifacts) and to provide contextual reasoning [12, 13] that complements traditional detection pipelines. Recent studies report promising results in tasks such as phishing understanding [14, 15], malware analysis support [16, 17], and intrusion-related reasoning [18, 19], although performance and reliability vary substantially with the evaluation setting and data realism [20, 21].

A practical advantage of LLM-based approaches is their flexibility in interfacing with multiple data sources, ranging from textual artifacts (e.g., emails, CTI reports) to telemetry summaries derived from logs and network events. Transformer-based architectures can model long-range dependencies in sequences, which is useful when correlating distributed evidence across time, sessions, or multi-step attack traces [22, 23].

However, despite their potential, LLM-based defenses face practical challenges, particularly limited interpretability [24] and substantial computational overhead (e.g., latency, energy, and deployment cost) [25, 26]. Additional concerns include latency and throughput constraints in high-volume monitoring, limited reproducibility and dataset realism, domain shift across organizations, and LLM-specific risks such as prompt injection, poisoned retrieval context, and unsafe tool/agent behaviors. [27, 28]

Accordingly, this survey focuses on *LLMs used for cyberattack detection and mitigation* across major attack families, including Denial of Service/Distributed Denial of Service (DoS/DDoS), phishing, malware, SQL injection (SQLi), cross-site scripting (XSS), eavesdropping, and Man-In-The-Middle (MITM) attacks. We structure the review around three complementary roles of LLMs: (i) semantic reasoning engines for understanding security artifacts, (ii) components within Intrusion Detection System/Security Operations Center (IDS/SOC) pipelines for detection, triage, and explanation, and (iii) controlled security agents that leverage retrieval and tools to support mitigation/response under safety constraints.

1.2 Motivations and contributions

The rapid advancement of LLMs is reshaping how defenders can interpret and act upon heterogeneous security signals. Modern security operations increasingly rely on diverse and unstructured inputs, including alerts, logs, incident reports, threat intelligence narratives, phishing content, and even code artifacts, while adversaries continuously adapt tactics to evade fixed signatures and rule-based pipelines. These conditions motivate the growing interest in LLMs as (i) semantic analyzers of unstructured security data, (ii) assistants for triage

and reasoning over context, and (iii) components within retrieval- and tool-augmented workflows for mitigation and response. At the same time, LLM-driven defense introduces new challenges, including robustness under adversarial inputs, real-time constraints, reproducibility concerns, and safe deployment in SOC environments. This survey examines LLM-based detection and mitigation across a spectrum of attacks, including DoS/DDoS, phishing, malware, SQL injection, XSS, eavesdropping, and MITM, and provides an analysis-oriented perspective that connects modeling choices, evaluation practices, and operational feasibility. The key contributions of this survey are summarized as follows:

- *Attack-centric taxonomy with detection/mitigation lens:* We propose and organize the literature using a unified taxonomy spanning major network and application layer attacks, separating detection from mitigation to clarify objectives, assumptions, and outcomes.
- *Unified comparative evidence study:* We provide a consolidated summary of representative LLM-based solutions (models, datasets, tasks, methods, and findings) and augment it with evaluation-relevant attributes (e.g., realism, assumptions, and reproducibility signals).
- *Analytical synthesis across studies:* Beyond summarizing the main contributions of recent works, we extract cross-paper insights on what works, where it fails, and why, including recurring design patterns (prompting, fine-tuning, RAG, tool/agent-based pipelines), modality requirements, and common failure modes across attack families.
- *Feasibility and safe deployment perspective:* We critically assess operational readiness (latency/throughput, integration with SIEM/SOAR/EDR, cost, maintenance, domain shift) and discuss LLM-specific risks (prompt injection, tool abuse, data poisoning, hallucinations), offering practical guidance for SOC adoption.
- *Open challenges and research directions:* We identify key gaps in datasets, metrics, benchmarking, robustness, and governance and outline concrete future directions for building reliable, secure, and deployable LLM-based cyber defense systems.

1.3 Review methodology and protocol

To strengthen rigor and reproducibility, we conducted this survey using a structured research-management workflow inspired by systematic scientific research planning and reporting practices [29]. Our objective is not only to collect prior work on LLM-driven cybersecurity, but to extract recurring pipeline designs, compare them under realistic SOC constraints, and derive research gaps and actionable directions.

1.3.1 Research Questions (RQs):

The survey is guided by four questions:

- **RQ1:** What recurring *LLM-enabled cybersecurity pipeline patterns* emerge across the literature (e.g., prompting, fine-tuned encoders, RAG-grounded reasoning, tool-using agents)?
- **RQ2:** Where do LLMs provide the most operational value (inline vs. analyst path), and what feasibility constraints dominate (latency/throughput, evidence/token budgets, cost)?
- **RQ3:** What LLM-specific risks arise in practice (prompt injection, poisoned retrieval, hallucinations, tool/agent misuse), and which assurance mechanisms are deployable?
- **RQ4:** How realistic are current evaluations (datasets, drift, imbalance, adversarial adaptation), and what benchmarking/reporting gaps remain?

1.3.2 Search strategy and sources

We queried major scholarly indices and digital libraries commonly used in cybersecurity and LLM research (e.g., Web of Science, IEEE Xplore, ACM Digital Library, Springer-Link, Elsevier/ScienceDirect, and arXiv). Search strings combined (i) LLM terms (“large language model,” “LLM,” “transformer,” “BERT/GPT,” “instruction tuning,” “RAG”), (ii) security tasks (“intrusion detection,” “phishing,” “malware,” “DDoS,” “SQL injection,” “XSS,” “SOC/SIEM/SOAR”), and (iii) safety/ops terms (“prompt injection,” “hallucination,” “tool use,” “agent,” “latency,” “privacy”).

1.3.3 Screening and eligibility

We included peer-reviewed papers and selected high-impact preprints that propose, evaluate, or systematize LLM-based methods for *detecting, analyzing, or mitigating* cyberattacks. We excluded works that (i) mention LLMs without a concrete security method or evaluation, (ii) are purely generic NLP without a cybersecurity task, or (iii) lack sufficient technical detail to extract pipeline design and evaluation characteristics. When multiple versions existed, we prioritized the most complete and recent version.

1.3.4 Data extraction and synthesis

For each included work, we extracted: threat family/task, evidence type, and representation (structured vs. free-form), model adaptation (prompting vs. fine-tuning/PEFT), grounding/tooling (RAG, knowledge graphs, SIEM pivots, agents), deployment placement (inline vs. analyst path), evaluation setting, and stated limitations/risks. We then mapped studies into our synthesis abstractions (Patterns and comparative Dimensions) to enable cross-paper comparison and to support the deployment readiness discussion in Section V.

1.3.5 Limitations

Given the rapid evolution of the field, some emerging contributions remain available only as preprints. Hence, we explicitly label publication status and emphasize reproducibility and evaluation realism when interpreting results. The structured protocol and explicit coding/synthesis steps are intended to improve transparency and reduce selection bias [29].

1.4 Survey organization

The remainder of this survey is organized as follows. Section I introduces the study, outlining the motivations and contributions. Section II reviews existing surveys on LLMs in cybersecurity and clarifies our scope and positioning relative to recent 2023–202 reviews. Section III provides the essential background needed to support the survey, including threat landscape, security artifacts, and LLM paradigms such as prompting, fine-tuning, retrieval-augmented generation, and tool/agent-based workflows). Section IV presents our taxonomy and analysis of LLM-based cyber defense methods under three complementary roles: (i) semantic reasoning engines, (ii) IDS/SOC pipeline components for detection, triage, and explanation, and (iii) retrieval- and tool-augmented security agents for mitigation/response under safety constraints. Section IV further separates detection from mitigation objectives and provides cross-paper synthesis within each attack family, covering DoS/DDoS, phishing,

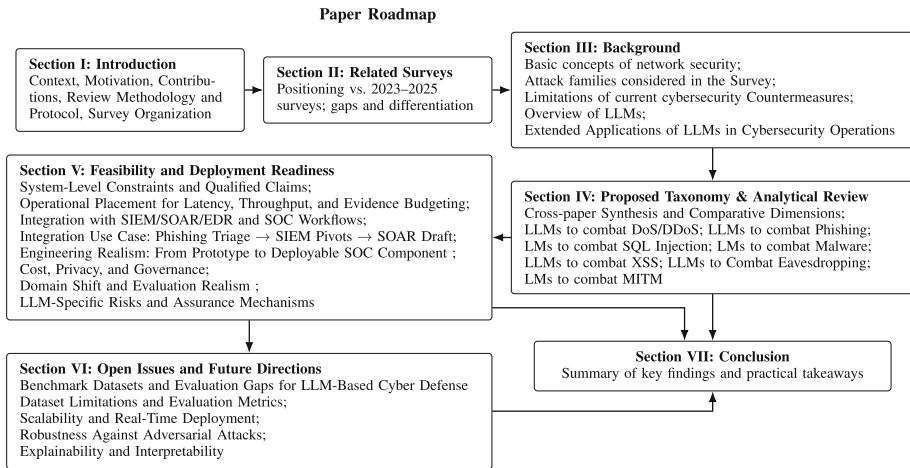


Fig. 1 Organization of the survey and roadmap of the main sections

malware, SQL injection, XSS, eavesdropping, and MITM. Section V evaluates feasibility and deployment readiness, discussing operational constraints (latency/throughput, integration with SIEM/SOAR/EDR, cost, domain shift) and summarizing LLM-specific risks and assurance mechanisms (e.g., prompt injection, poisoned retrieval, hallucinations, and tool/agent misuse). Section VI explores open issues and future research directions in LLM-driven cybersecurity. Finally, Section VII concludes the survey by summarizing the key findings and highlighting practical takeaways for researchers and practitioners. This organization is illustrated in Fig. 1.

2 Related surveys

The rapid uptake of LLMs in cybersecurity has triggered a growing body of survey work that reviews (i) defensive uses of LLMs for detection, triage, and response, and (ii) security and trust risks introduced by LLM-enabled pipelines. In this section, we position our survey relative to recent 2023–2025 reviews and clarify our added value beyond “broader attack coverage” by emphasizing a detection–mitigation separation, cross-paper analytical synthesis, feasibility constraints, and LLM-specific risk/assurance analysis.

In 2024, Chen et al. provided an application-oriented overview of how LLMs can be used for cyber threat detection, covering tasks and representative approaches across the detection pipeline [30]. Da Silva and Westphall surveyed LLM use in cybersecurity more broadly and discussed limitations relevant to practical adoption, including reliability and operational constraints [31]. Hassanin and Moustafa presented a comprehensive overview of LLMs for cyber defenses, highlighting opportunities across multiple security functions and outlining future directions [32]. Complementarily, Yao et al. focused on security and privacy issues of LLMs themselves, surveying threats such as data leakage and prompt-based attacks, which are directly relevant to the safe use of LLMs in defender pipelines [33]. Kheddar surveyed Transformers and LLMs for intrusion detection systems, providing IDS-centric coverage that is valuable for network telemetry use cases [34].

In 2025, survey work further expanded into more operational and SOC-centric perspectives. Habibzadeh et al. reviewed LLMs for SOC, emphasizing how LLMs can assist triage, summarization, and decision support in analyst workflows [35]. Liu et al. discussed the dual role of LLMs in network security and summarized emerging research trends [16]. Feng and Sakurai reviewed the evolution of network intrusion detection toward LLM collaboration and highlighted emerging risks, which inform the feasibility and assurance discussion [8]. For malware-specific pipelines, Guo et al. surveyed malware analysis with LLMs and summarized emerging directions at the intersection of code/artifact understanding and security analytics [17].

Despite these advances, important gaps remain. First, many surveys did not explicitly separate *detection* from *mitigation/response*, even though these stages impose different constraints, evaluation criteria, and safety implications. Second, feasibility aspects were often under-specified (e.g., latency/throughput constraints, integration with Security Information and Event Management (SIEM)/Security Orchestration, Automation, and Response (SOAR)/Endpoint Detection and Response (EDR) stacks, reproducibility signals, and maintenance under drift). Third, LLM-specific risks (prompt injection, poisoned retrieval, hallucinations, and tool/agent misuse) are frequently discussed in isolation rather than mapped to concrete defender workflows. Our survey addresses these gaps by organizing the literature around an attack-centric taxonomy while explicitly separating detection from mitigation/response, extracting cross-paper design patterns and failure modes, and providing feasibility and assurance criteria for safe IDS/SOC integration.

Tables 1 and 2 summarize representative 2024–2025 surveys and contrast them with our survey from two complementary perspectives. Table 1 provides a compact positioning analysis using expanded criteria (e.g., analytical synthesis, feasibility/SOC integration, and risk/assurance), while Table 2 reports a separate attack family coverage matrix to highlight the breadth of addressed threat types.

3 Background

3.1 Basic concepts of network security

Network security refers to a comprehensive framework of strategies, policies, technologies, and practices designed to safeguard the confidentiality, integrity, and availability of data and systems across a network. It encompasses preventive, detective, and responsive controls that reduce the likelihood and impact of both external and insider threats while maintaining operational continuity. This makes it a critical component in today's increasingly digital and interconnected world, where challenges such as cyberattacks, data breaches, and unauthorized access are prevalent [36].

The core security objectives are confidentiality, integrity, and availability (CIA), often complemented by authentication and accountability/non-repudiation, which together define the expected trust properties of networked systems [37–39]. From an operational perspective, these objectives are commonly implemented through risk-based security programs and continuous monitoring practices (e.g., incident handling and response loops), as reflected in widely adopted frameworks such as NIST guidance [40–42].

Network security is an ongoing process that typically spans risk analysis [43, 44], policy definition [45], control implementation [46], and continuous monitoring/incident response [47], forming a feedback loop that supports resilience [48] and adaptability. To illustrate

Table 1 Comparison of related surveys (2024–2025)

Ref	Year	Scope/Focus	D	M	AC	AS	FS	RA
[30]	2024	LLMs for cyber threat detection (tasks, methods, datasets)	✓	–	partial	partial	partial	partial
[31]	2024	Broad survey of LLM applications in cybersecurity	✓	partial	–	partial	partial	partial
[32]	2024	LLMs for cyber defenses: opportunities and directions across tasks	✓	partial	–	partial	partial	partial
[33]	2024	LLM security & privacy (threats, vulnerabilities, mitigations)	–	–	–	partial	–	✓
[34]	2024	Transformers/LLMs for IDS (network intrusion detection)	✓	–	partial	✓	✓	partial
[35]	2025	LLMs for SOC: analyst workflows, triage, operational considerations	✓	partial	–	partial	✓	partial
[16]	2025	Survey and trends on the dual role of LLMs in network security	✓	partial	–	partial	partial	partial
[8]	2025	NIDS evolution toward LLM collaboration and emerging risks	✓	–	partial	partial	✓	✓
[17]	2025	Malware analysis with LLMs (artifact/code-aware pipelines)	✓	partial	partial	partial	partial	partial
Our Survey	--	Attack-centric taxonomy with explicit detection vs mitigation separation; analytical synthesis; feasibility & assurance for SOC/IDS adoption	✓	✓	✓	✓	✓	✓

Legend: D = Detection; M = Mitigation/Response; AC = Attack-centric; AS = Analytical synthesis; FS = Feasibility & SOC integration; RA = LLM Risk & Assurance. **Marks:** ✓ = explicitly addressed; partial = partially addressed; – = not addressed/out of scope

Table 2 Attack coverage matrix across surveys (2024–2025)

Ref	Year	DoS/DDoS	Phishing	Malware	SQL i	XSS	Eaves-drop	MITM
[30]	2024	partial	✓	✓	partial	partial	–	partial
[31]	2024	partial	✓	✓	partial	partial	partial	partial
[32]	2024	partial	✓	✓	partial	partial	partial	partial
[33]	2024	–	–	–	–	–	–	–
[34]	2024	✓	partial	partial	partial	–	–	partial
[35]	2025	partial	partial	partial	partial	–	–	partial
[16]	2025	partial	partial	partial	partial	partial	partial	partial
[8]	2025	✓	partial	partial	partial	–	–	partial
[17]	2025	–	–	✓	–	–	–	–
Our Survey	2026	✓	✓	✓	✓	✓	✓	✓

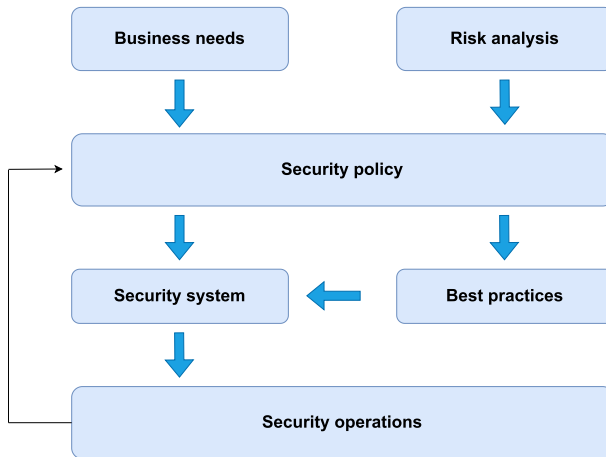


Fig. 2 Network security system lifecycle [49]

this, Fig. 2 presents the lifecycle of a network security system. It begins with identifying business needs and conducting a risk analysis, which serve as the foundation for establishing a security policy. This policy then guides the implementation of a security system and the adoption of best practices. The process culminates in ongoing monitoring and maintenance through security operations, creating a continuous feedback loop to ensure adaptability and resilience.

3.2 Attack families considered in the survey

To support the taxonomy in Section IV below, we briefly summarize the attack families covered in this survey and highlight the typical observable artifacts used by detection and mitigation pipelines (e.g., flows/packets, logs, emails/web content, and code-level traces). This section is not intended to be exhaustive, but rather provides only the minimum context required to interpret the surveyed LLM-based methods.

3.2.1 Denial of service/distributed denial of service

By flooding resources with malicious requests, denial of service (DoS) and distributed denial of service (DDoS) attacks aim to render a network or service unavailable. DDoS commonly relies on distributed sources (e.g., botnets), which makes attribution and filtering more challenging and places strong constraints on detection latency and throughput [50, 51]. In practice, evidence may appear in volumetric traffic shifts, anomalous request patterns at the application layer, and correlated spikes across distributed vantage points, which influence how LLM-based approaches can be integrated (often as reasoning/triage components rather than inline real-time filters).

3.2.2 Malware

Malware, or malicious software, is software created expressly to damage, take advantage of, or access computer systems without authorization. It includes diverse families (e.g., trojans,

ransomware, worms) and is commonly analyzed using *static*, *dynamic*, or *hybrid* methods. *Static analysis* inspects an executable without running it (e.g., binary code, opcodes/strings, imported libraries, permissions, and other file-level features). *Dynamic analysis* observes the program while it executes in a controlled environment (e.g., API/system-call traces, system logs, network activity, and runtime behavioral sequences). *Hybrid analysis* combines static features with runtime observations to improve robustness and detection coverage. LLM-based defense methods in this area frequently leverage semantic reasoning over reports and labels, code-aware representations, or retrieval over threat intelligence to assist detection, triage, and response. Well-known ransomware outbreaks illustrate the operational impact of malware and motivate robust detection and response workflows [52–55].

3.2.3 SQL injection

SQL Injection [56, 57] is a type of cyberattack where malicious SQL code is injected into input fields to take advantage of flaws in programs that communicate with databases. It typically exploits insufficient input validation and unsafe query construction, and defenses commonly rely on secure coding practices (e.g., parameterized queries) and application monitoring [58, 59]. For surveys of LLM-based methods, SQLi is especially relevant because detection can involve both natural-language-like inputs (payload patterns) and code/context understanding in web backends.

3.2.4 Phishing:

Phishing [60–62] is a kind of social engineering assault in which the attacker poses as a reliable organization in order to trick victims into disclosing private information. Because phishing content is inherently linguistic and contextual (tone, intent, persuasion strategies, and brand imitation), it is a natural fit for LLM-assisted analysis of emails, URLs, landing pages, and conversation traces [63].

3.2.5 Man-in-the-middle

A man-in-the-middle attack [64–66] refers to a class of cyberattacks in which an adversary surreptitiously places itself between two legitimate communicating parties without their knowledge. Observable signals often arise from protocol anomalies, certificate and handshake inconsistencies, or suspicious routing behaviors, which can be combined with logs and contextual evidence in detection pipelines [67].

3.2.6 Eavesdropping

In cybersecurity, eavesdropping [68–71] is the term used to describe the illegal monitoring and interception of data over a network. It may be passive (silent interception) or active (modification/injection), and commonly targets confidentiality by capturing credentials, session identifiers, or sensitive payloads [72].

3.2.7 Cross-site scripting

Cross-site scripting (XSS) is a web security vulnerability that lets attackers inject malicious scripts into trusted websites. XSS detection may rely on payload patterns, contextualized code

analysis, and web telemetry, while mitigation typically emphasizes robust input validation, output encoding, and browser-side protections [73].

In the remainder of the survey, we use these attack families to structure a taxonomy of LLM-based approaches and to contrast detection-oriented pipelines with mitigation/response workflows under realistic operational constraints.

3.3 Limitations of current cybersecurity countermeasures

Although network defense has matured considerably [74, 75], widely deployed countermeasures still face persistent limitations when operating under realistic conditions (high-volume telemetry, heterogeneous assets, and adaptive adversaries). In practice, many deployments remain anchored in signature- and rule-driven detection logic. These methods provide strong detection performance for known patterns with low overhead, but they are inherently brittle against evolving attack surfaces (e.g., zero-day behaviors, obfuscation, and polymorphism). Moreover, the continuous maintenance required to keep policies and signatures current introduces operational friction and unavoidable coverage gaps.

Data-driven approaches, i.e., machine learning (ML)-based and deep learning (DL)-based IDS, partially improve generalization by learning statistical regularities from flows, logs, and application events. However, their performance is often sensitive to dataset bias, class imbalance, and distribution shift. Recent IoT-focused IDS studies and comparative evaluations, including a 2024 review and multiple 2023 analyses of boosting- and tree/ensemble-based detectors for IoT intrusion and botnet traffic [76–79], highlighted both the practical gains and the persistent sensitivity of learned decision boundaries to data quality and changing traffic baselines. Concept drift (changing traffic baselines, new services, evolving user behavior) can degrade decision boundaries over time, increasing false positives and undermining alert reliability. This is especially problematic for *SOC feasibility*, where excessive alert volumes translate directly into analyst fatigue and delayed triage.

Prior deep learning IDS surveys and systems, spanning convolutional neural networks (CNN)/ recurrent neural networks (RNN) sequence models, graph-based variants, and autoencoder-style anomaly detection, have established strong baselines for learning from numeric flow features, packet representations, and time-series telemetry. These approaches are typically *efficient at inference* (once feature pipelines are fixed) and can be deployed as compact classifiers or detectors, but they often require *task-specific feature engineering*, offer limited semantic interpretability, and degrade under domain shift unless continuously retrained or calibrated. In contrast, LLM-based defenses trade some of this efficiency for *semantic generality*: they can ingest heterogeneous artifacts (alerts, logs, tickets, email/web content) and support analyst-facing outputs (summaries, evidence narratives, and query/playbook drafting). However, LLM pipelines introduce distinct limitations that are less emphasized in conventional DL surveys, notably token/latency cost, prompt/retrieval attack surfaces (prompt injection, poisoned retrieval), and the need for strict evidence schemas and governance when integrated with SOC tooling. This comparison clarifies our survey's unique contribution, i.e., not replacing established DL-based IDS, but analyzing *where LLMs add value* (triage, grounding, workflow integration) and *what new risks and constraints* they introduce at deployment time.

A second major bottleneck concerns *Risk & Assurance*. Many deep models behave as black boxes and offer limited transparency into *why* a session, payload, or trace is flagged. This restricts analyst trust, complicates root-cause analysis, and weakens the defensibility of mitigation actions in regulated or high-stakes environments. Recent IDS research has high-

lighted the importance of embedding explainability mechanisms (e.g., post hoc attribution and local explanations) to improve operational adoption and auditability [80]. Yet, explainability remains inconsistently supported across current pipelines and is rarely integrated end-to-end with response logic.

Scalability and time-to-decision further constrain real-world deployments. Indeed, modern networks generate high-velocity streams of multi-source telemetry while pipelines, which require heavyweight feature extraction or deep inference, can struggle to meet latency budgets. Consequently, systems are often forced into coarse filtering, sampling, or offline analysis, which reduces effectiveness for time-critical scenarios (e.g., fast-moving DDoS, lateral movement). These constraints create a gap between academic evaluation and deployable *SOC-ready* solutions.

Finally, existing defenses frequently over-emphasize *Detection* while providing limited support for *Mitigation/Response*. Response is commonly handled through fragmented tools and manual playbooks, requiring analysts to correlate alerts, validate evidence, and trigger containment actions across disparate systems. Standard operational guidance emphasizes iterative monitoring and incident response loops [42], but practical implementations remain weakly integrated, particularly for multi-step, campaign-style attacks that require contextual reasoning and coordinated actions.

These limitations motivate next-generation frameworks that jointly address the dimensions summarized in Tables I–II, specifically, improving *Detection* robustness under drift and adversarial adaptation, enabling verifiable *Mitigation/Response* orchestration, maintaining *SOC feasibility* under scale and latency constraints, and strengthening *Risk & Assurance* via transparency, accountability, and evidence-backed decision support.

3.4 Overview of large language models

Large Language Models (LLMs) are Transformer-based models that learn rich linguistic and semantic representations from large-scale corpora, enabling strong performance on tasks such as extraction, classification, summarization, and structured reasoning over text [81, 82]. In cybersecurity, these capabilities are most relevant when security-relevant evidence is expressed as natural language or semi-structured text, such as incident tickets, threat intelligence reports, alert descriptions, email bodies, URLs, and code-related artifacts [83–85]. Importantly, our survey focuses on *LLMs used for cyberattack detection and mitigation*, not on generic NLP applications.

3.4.1 Operational roles in cyber defense:

Across the literature reviewed in this survey, LLMs are typically deployed in three complementary roles:

- *Semantic reasoning engines* [86, 87] that interpret unstructured security artifacts (e.g., phishing emails, CTI narratives, vulnerability descriptions) and produce structured insights useful for analysts.
- *IDS/SOC pipeline components* [6, 8, 35, 88–92] that support detection, triage, and explanation by correlating evidence across logs, alerts, and contextual documentation.
- *Retrieval- and tool-augmented security agents* [93–95] that assist mitigation/response (e.g., proposing containment steps or playbook actions) under controlled execution and auditing constraints.

3.4.2 Adaptation and grounding paradigms

The surveyed works primarily rely on three families of techniques to adapt LLMs to security tasks:

- *Prompting (zero-/few-shot)* instructing the model via carefully designed prompts and exemplars to perform classification, extraction, or explanation without changing model parameters. [96–98]
- *Fine-tuning (including parameter-efficient tuning)* [99, 100] adapting the model to domain-specific distributions (e.g., phishing corpora, security logs, exploit descriptions) when labeled data and compute allow.
- *Retrieval-Augmented Generation (RAG)* grounding the model's outputs with retrieved external knowledge (e.g., CTI repositories, incident runbooks, vulnerability databases, SOC playbooks), improving traceability and reducing reliance on internal memorization. [9, 101–103]

RAG is particularly attractive for security because it enables updated knowledge and audit-friendly citations, but it also introduces new risk channels [104] such as poisoned retrieval content and instruction injection through untrusted documents.

3.4.3 Tool use and agentic workflows

Beyond text generation, LLM systems increasingly incorporate external tools (e.g., parsers, query engines, sandbox analyzers) and multi-step reasoning policies to iteratively gather evidence, check hypotheses, and refine decisions. In cyber defense, this paradigm aligns naturally with triage and response workflows, where actions must be justified, constrained, and logged. However, it requires strict guardrails to prevent unsafe recommendations or unauthorized tool execution.

3.4.4 Multimodal LLMs in security

Some recent studies, e.g., [13, 105–107], extended LLMs to multimodal settings (MLLMs) by integrating additional modalities such as images, audio, or user interface (UI) screenshots. In security contexts, multimodality can be relevant for tasks such as analyzing screenshots of phishing pages, UI-based deception, or visual indicators within incident artifacts. Because the majority of network security telemetry remains text- or log-centric, we keep MLLMs as a secondary topic and discuss them when they materially affect detection/mitigation design choices.

3.4.5 Deployment's limitations and risks

For real-world adoption, the surveyed literature highlights recurring limitations that we treat as first-class evaluation dimensions throughout the survey:

- *Reliability* Model hallucinations, overconfident explanations, and sensitivity to prompt phrasing or context selection.
- *Feasibility* computational overhead, latency/throughput constraints for high-volume monitoring, and maintenance under concept drift and domain shift.
- **Security of the defense pipeline:** prompt injection and instruction hijacking (especially in analyst copilots), poisoned retrieval contexts (RAG), and risks from agentic tool use.

- *Governance* privacy and compliance constraints, auditability, and reproducibility gaps across datasets and evaluation protocols.

Accordingly, in later sections, we compare LLM-based methods not only by task accuracy, but also by evaluation realism, operational constraints, and assurance mechanisms required for safe IDS/SOC integration.

3.5 Extended applications of large language models in cybersecurity operations

3.5.1 Large language models-based code and vulnerability analysis

LLMs are increasingly applied to software security tasks that complement network-level defense, particularly vulnerability discovery, triage, and remediation support. Code-aware LLMs and instruction-tuned models can analyze source code, configuration files, commit histories, and vulnerability descriptions to identify insecure coding patterns, misconfigurations, and logic flaws. Typical applications include mapping code segments to known weakness categories, prioritizing vulnerability alerts based on contextual risk, and summarizing patch rationale from security advisories and version-control diffs.

Compared to traditional static analysis tools that rely on handcrafted rules and syntactic signatures, LLMs offer semantic reasoning capabilities that capture higher-level intent and data-flow relationships. This enables more interpretable vulnerability explanations and improved prioritization for remediation workflows. However, practical deployment remains constrained by limitations such as hallucinated fixes, incomplete reasoning over large code-bases, and sensitivity to prompt formulation and context window size.

3.5.2 Large language models for penetration testing assistance

LLMs are also being explored as assistants in penetration testing and red-team workflows. Rather than replacing expert-driven assessments, LLMs can support planning, documentation, and reasoning tasks by translating high-level testing objectives into structured assessment plans, generating contextual payload templates, and correlating tool outputs into coherent attack narratives. In controlled environments, LLMs can reduce manual overhead by automating repetitive tasks such as parameter adaptation, report drafting, and vulnerability explanation. They can further assist in mapping discovered weaknesses to exploitation paths and recommending follow-up tests based on observed system configurations. This capability improves workflow efficiency and reporting consistency across large-scale security assessments.

However, the use of generative models in offensive security contexts introduces governance and safety concerns. Automated exploitation without appropriate safeguards can amplify misuse risks, violate testing scopes, or generate unsafe artifacts. Consequently, responsible deployment requires strict access control, sandboxed execution, audit logging, and alignment with organizational authorization policies. These constraints reinforce the importance of controlled, supervised integration rather than fully autonomous red-team agents.

3.5.3 Large language models-enabled security orchestration and automated response

Security orchestration and automated response represent a critical layer where LLMs can enhance operational effectiveness. Modern SOC's operate on heterogeneous alert streams

originating from IDS, EDR, cloud monitors, application logs, and endpoint telemetry. LLMs can assist by correlating alerts across sources, enriching incidents with contextual explanations, and recommending response playbooks based on historical cases and organizational policies.

Unlike static rule-based automation, LLM-assisted SOAR enables adaptive reasoning over incomplete and ambiguous evidence, supporting semi-automated containment decisions such as traffic blocking, host isolation, credential revocation, and ticket escalation.

Nevertheless, response automation imposes strict reliability and accountability requirements. Incorrect mitigation actions may disrupt critical services or introduce secondary failures. As a result, practical deployments often favor human-in-the-loop architectures, where LLMs provide decision support and prioritization, while final execution remains subject to analyst validation. Integration with existing SIEM and SOAR platforms, policy enforcement layers, and continuous monitoring of action outcomes is therefore essential for SOC feasibility.

3.5.4 LLMs for threat intelligence processing and knowledge integration

Threat intelligence analysis [108] is another domain where LLMs demonstrate strong potential due to their ability to process unstructured textual data. Security teams routinely ingest large volumes of threat reports, advisories, vulnerability disclosures, and incident summaries. LLMs can assist by extracting entities and relations, normalizing indicators of compromise, and generating structured knowledge representations from heterogeneous sources. When combined with retrieval-augmented generation, LLMs enable interactive investigation workflows in which analysts can query historical campaigns, correlate indicators across repositories, and summarize emerging attack trends. This capability improves situational awareness and accelerates incident context building, particularly in fast-moving threat landscapes.

4 Proposed taxonomy and analytical review

This section introduces a taxonomy that goes beyond attack-type grouping by capturing *how* LLMs are instantiated in security pipelines and *under which operational constraints* they are effective. We explicitly separate (i) architectural design patterns recurring across papers, (ii) reasons behind success/failure in realistic settings, and (iii) comparative dimensions that enable cross-paper evaluation independent of the targeted attack class. This organization supports a practitioner-oriented reading: the same LLM strategy may perform well in offline forensics yet be infeasible for inline IDS, and conversely, a high-throughput detector may underperform on semantic deception where LLM reasoning is beneficial.

Throughout this section, we use “LLM” broadly to include instruction-tuned foundation models (e.g., GPT-style and LLaMA-style), as well as security-specialized Transformer encoders (e.g., BERT variants trained/adapted for security telemetry) when they are used as language-driven reasoning or representation modules [109, 110]. We also account for retrieval-augmented and tool-using systems, since security deployments often require grounding in external knowledge (CTI reports, playbooks, logs) and controlled tool execution [111–113].

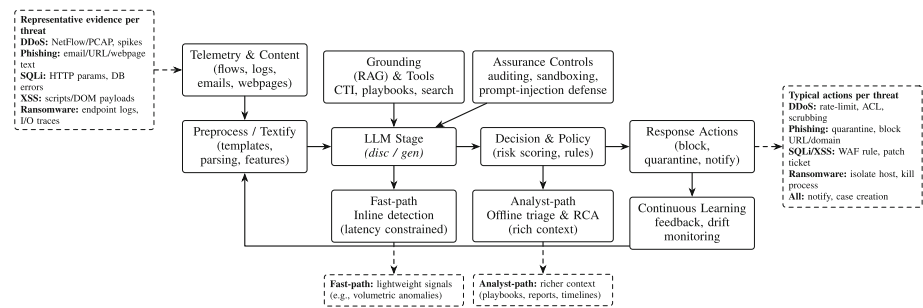


Fig. 3 Pipeline for LLM-based cyberattack detection and mitigation, highlighting (i) fast path vs. analyst path, (ii) generative vs. discriminative roles, and (iii) assurance controls, with representative threat-specific evidence and response mappings

Fig. 3 provides the end-to-end reference pipeline used throughout Section IV to situate and compare LLM-based approaches for attack detection and mitigation, from telemetry acquisition and representation to reasoning, decision, and response execution.

4.1 Cross-paper synthesis and comparative dimensions

A core limitation of many early surveys is that they remain descriptive, making it difficult to extract design principles that generalize across datasets, attacks, and deployment contexts. We therefore synthesize the literature using two complementary lenses: (L1) *architectural patterns* that describe how LLMs interface with security telemetry and tooling, and (L2) *comparative dimensions* that capture feasibility trade-offs (latency, cost, robustness, and integration). The result is a taxonomy that can be applied consistently across attack types, enabling cross-paper comparison even when evaluation setups differ.

4.1.1 Common architectural patterns

We observe five recurring architectural patterns that appear across DoS/DDoS, phishing, malware analysis, and web-attack detection studies:

a) P1: Telemetry “textification” and prompt-based reasoning: Raw artifacts (packets, flows, HTTP traces, logs, emails) are converted into structured textual descriptors (e.g., key–value summaries, tokenized sequences, templated prompts) and analyzed via prompting or lightweight adaptation. This pattern is attractive for rapid deployment and explainability, but is sensitive to representation choices and context window limits, especially under high-volume streams [114, 115].

b) P2: Security-specialized discriminative encoders: Rather than using a fully generative model, many systems fine-tune or design compact Transformer encoders for classification over fixed-length security representations (flows, syscalls, encoded packets). This improves throughput and stability for inline IDS and edge/IoT settings and can incorporate privacy-preserving encodings [116].

c) P3: Retrieval-augmented security reasoning (RAG): LLM outputs are grounded by retrieving external knowledge such as CTI repositories, vulnerability databases, incident reports, and SOC playbooks. RAG can reduce stale knowledge and improve traceability, but it introduces additional attack surfaces (poisoned retrieval content, instruction injection via retrieved documents), requiring robust sanitization and provenance control [111].

Table 3 Cross-paper architectural patterns (see table 3) observed in LLM-based detection and mitigation

Pattern	Description	Observed strengths/failure modes
Textification of telemetry	Convert packets/flows/logs/syscalls/HTML into structured text prompts or token sequences	Strength: enables semantic reasoning and long-range context; Risk: lossy encoding can hide timing/protocol cues
Agentic/tool-augmented SOC	LLM plans multi-step workflows (triage, correlation, enrichment, reporting) with tools	Strength: analyst productivity, evidence linking; Risk: requires strict execution control and auditability
Multimodal fusion	Combine visual + textual + structural cues (e.g., phishing webpages) with MLLMs	Strength: better coverage of heterogeneous signals; Risk: higher compute cost and dataset complexity
Hybrid fast path + LLM stage	Classical ML/signatures for filtering + LLM for second-stage semantics	Strength: balances latency and interpretability; Risk: integration complexity and error propagation

d) P4: Tool-using/agentic pipelines for triage and response: LLMs are embedded in multi-step workflows that plan, call tools (parsers, SIEM queries, sandboxes), and refine conclusions. This maps naturally to SOC processes (alert summarization, evidence correlation, response drafting), but must be paired with strict guardrails, auditing, and least-privilege execution [112, 113].

e) P5: Multimodal fusion for encrypted or heterogeneous signals: When payload visibility is limited (e.g., encrypted traffic) or signals are heterogeneous (HTML+visual cues, packet timing+metadata), several works adopt multimodal alignment or fusion objectives to preserve discriminative information and improve interpretability [107].

4.1.2 Why approaches succeed or fail

Reported performance gains often depend less on “the LLM” and more on pipeline assumptions. We distill five dominant factors that explain success/failure across papers:

- 1) *Representation fidelity* Textification can discard timing/volume cues critical for DoS/DDoS, while fixed encodings may miss semantic deception cues in phishing and social engineering.
- 2) *Evaluation realism* Offline experiments on curated datasets may not reflect SOC conditions (concept drift, alert imbalance, encrypted traffic, adversarial adaptation).
- 3) *Robustness and adversarial pressure*: LLM-facing interfaces create exposure to prompt injection, instruction hijacking, and poisoning via external knowledge sources, impacting both detection and mitigation reliability [117].
- 4) *Operational constraints* Throughput, latency, and cost determine whether an approach can be deployed inline, used for batch hunting, or restricted to analyst copilot functions.
- 5) *Action safety for mitigation* Agentic designs can draft playbooks and recommend actions, but require constrained tool access, human approval gates, and post-incident audit trails to prevent unsafe behavior [118].

These factors motivate our cross-paper comparison, i.e., each surveyed method is evaluated not only by task metrics (e.g., accuracy/F1), but also by feasibility assumptions (e.g.,

Table 4 Comparative dimensions used to synthesize LLM-based cyber defense methods (applied across attack families)

Dimension	What to report	Typical implications
Latency vs. accuracy	Runtime regime (streaming vs. batch), throughput assumptions, model size/context, preprocessing cost	Inline IDS favors compact/efficient models; offline analysis permits larger context and richer reasoning
Generative vs. discriminative	Classifier/anomaly score vs generation (explanations, rules, playbooks)	Generative adds interpretability and analyst utility but needs stronger guardrails and calibration
Inline vs. offline	Placement in pipeline (fast path filtering vs analyst path triage)	Many “wins” occur in analyst path even when inline deployment is infeasible
Grounding & tools	RAG/tool use/sandboxes; evidence traceability	Improves freshness and traceability; introduces injection/poisoning surfaces
Assurance	Monitoring, auditing, constraints, and robustness measures	Required for SOC adoption and risk containment

data availability, inference budget, integration complexity) and assurance mechanisms (e.g., guardrails, auditing, fallback strategies).

4.1.3 Comparative dimensions

To systematically compare heterogeneous proposals, we introduce three orthogonal comparative dimensions (see Table 4 that recur across Section IV):

a) D1: Inline versus offline deployment Inline detection operates under tight latency/throughput constraints (e.g., line-rate monitoring), favoring compact discriminative encoders or two-stage cascades (fast filter + deeper reasoning). Offline analysis (threat hunting, forensics) can exploit longer contexts and richer tool access, favoring prompting, RAG, and agentic workflows.

b) D2: Latency versus accuracy (and cost) LLM-based pipelines often improve semantic interpretation and explanation, but can increase inference latency and cost. A common pragmatic solution is staged processing: classical/compact models handle the bulk stream, while LLMs analyze the minority of suspicious cases in depth, improving overall SOC usability without breaking budgets [119].

c) D3: Generative versus discriminative use Generative use typically supports explanation, summarization, rule drafting, and reasoning over heterogeneous evidence; discriminative use targets stable classification and anomaly scoring. Hybrid deployments frequently combine both: discriminative models trigger alerts, while generative models produce analyst-facing narratives, triage rationales, and mitigation drafts.

4.1.4 Large language models versus classical/ML-based IDS

Classical/ML-based IDS systems (signatures, statistical anomaly detection, supervised ML/DL classifiers) remain strong baselines for high-throughput detection, particularly when attack patterns are well captured by flow features or signatures. However, the surveyed literature indicates that LLMs add distinctive value in four settings, namely

- **Semantic-rich artifacts:** phishing emails, incident tickets, threat reports, and heterogeneous logs where meaning, intent, and context matter.
- **Cross-source correlation:** synthesizing evidence across alerts, logs, and external CTI/playbooks, especially under partial observability.
- **Explainability and analyst workflow:** generating concise rationales, summarizing incidents, and producing actionable remediation narratives that reduce cognitive load.
- **Rule/signature authoring and maintenance:** assisting with the generation/refinement of IDS rules and detection logic, improving maintainability and response speed [119].

Conversely, we emphasize that LLMs are *not* universally superior. For instance, for line-rate DoS/DDoS screening, classical detectors or compact discriminative Transformers often remain more feasible, and LLMs are best positioned as a second-stage reasoning/triage layer. This “LLM-in-the-loop” positioning also reduces risk under adversarial manipulation by limiting the LLM’s authority and constraining tool execution.

4.2 LLMs to combat DoS/DDoS

DoS and DDoS attacks remain among the most disruptive threats in operational networks, and their detection/mitigation is increasingly challenged by traffic encryption, high telemetry volume, and rapidly evolving tactics. In the recent literature, LLMs are not used as a single “drop-in IDS” model. Instead, they appear as (i) *semantic reasoning layers* over textified telemetry, (ii) *domain-specialized discriminative encoders* for flow/packet representations, and (iii) *agentic components* that assist mitigation and operational decision-making. Accordingly, we organize DoS/DDoS works around the *architectural patterns* and *deployment comparative dimensions*.

4.2.1 Detecting DoS/DDoS

a) Textification + LLM reasoning (often offline/semi-online): Several works convert network artifacts into textual forms and then apply LLM prompting or lightweight adaptation for DoS/DDoS recognition. For instance, Llama-based pipelines report strong detection metrics when packet/flow features are rendered into text prompts [114]. However, success depends on (i) whether the text representation preserves discriminative timing/volume cues, and (ii) whether the deployment can tolerate higher inference latency compared with classical IDS. DrLLM-style designs further embed global/local context into a structured reasoning pipeline to improve robustness in low-supervision or zero-shot regimes [115, 120].

b) Lightweight security encoders (inline): In contrast to prompt-heavy LLM pipelines, SecurityBERT targets resource-constrained IoT/IIoT settings using privacy-preserving fixed-length encodings and a compact Transformer model, emphasizing deployability (low overhead) while maintaining high detection performance across multiple attack types, including DoS/DDoS [116]. This line of work is typically stronger on *latency/throughput feasibility*, but provides less flexible “open-ended” reasoning than generative LLM setups.

c) Multimodal/encrypted-traffic anomaly detection: Encrypted traffic weakens payload-based heuristics and motivates representation learning that fuses packet-level signals with language-style objectives. PACKETCLIP combines packet representations with language-aligned contrastive pretraining and hierarchical reasoning to improve both detection and interpretability under constrained labels [107]. Compared with pure textification, multimodal alignment can reduce information loss, but typically increases training complexity and may require careful evaluation under realistic encryption and traffic-mix conditions.

d) LLMs for signature/rule synthesis (hybrid with IDS): Another pragmatic trend is to keep classical IDS engines for fast matching while using LLMs to synthesize or refine human-readable rules/signatures from observed artifacts. Hex-to-signature pipelines demonstrate that LLMs can assist in generating Suricata-style rules and explanations, improving maintainability and analyst workflow [119]. Related work on LLM-assisted regular-expression generation further supports the same “LLM for rule authoring” angle, which is operationally valuable but must be validated for correctness and false-positive control [121].

e) Semantic logs and application layer DoS: Some recent efforts detect DoS-like behaviors from event/transaction logs when source code is unavailable (e.g., smart-contract ecosystems), where LLMs interpret sequences of events and infer denial patterns [122]. While these settings differ from conventional network DDoS, they strengthen the broader claim that LLMs are most helpful when detection depends on *semantic interpretation* of complex logs rather than only on low-level traffic statistics.

4.2.2 Mitigating DoS/DDoS

a) Agentic mitigation and decision support Mitigation-focused work increasingly frames LLMs as coordination layers that translate detected conditions into response actions, policies, or role-specific recommendations. ShieldGPT exemplifies this direction by combining traffic representation with domain knowledge injection and role-based response generation to improve mitigation effectiveness in simulated DDoS settings [118]. In practice, these systems are best positioned as “human-in-the-loop copilots” unless they incorporate strict guardrails, audit logs, and safe-action constraints.

b) LLMs as *targets* of DoS and implications for defense An important parallel thread shows that LLM-integrated security systems can themselves be degraded via DoS-style behaviors (e.g., inducing excessive safeguard rejections), which highlights the need to treat *availability* and *robustness* as first-class evaluation criteria for LLM-enabled defense pipelines [123]. This motivates mitigation designs that include rate limiting, workload shaping, and fallbacks to classical detectors under overload, especially for SOC-scale alert volumes.

4.2.3 Benefits for IDS/SOC integration

DoS/DDoS studies suggest that inline, high-throughput environments favor specialized discriminative encoders and hybrid designs (fast filter + LLM triage), whereas offline investigation benefits more from LLM reasoning, explanation, and rule synthesis.

4.2.4 Open research gaps

DoS/DDoS defense is fundamentally constrained by *real-time* requirements (tight latency budgets, high event rates, bursty traffic) that many LLM-based studies do not report or benchmark under realistic load. A key gap is *throughput feasibility*: token-heavy representations and long-context reasoning can be incompatible with line-rate detection unless used in staged designs (fast path filtering + slow path semantic triage). Additionally, *continuous updating* is often overlooked: DDoS traffic patterns shift quickly due to changing botnets and attack tooling, requiring online calibration, drift monitoring, and safe fallback behavior under surge conditions. Future work should report end-to-end latency/throughput (p95/p99), cost per event, and demonstrate safe degradation and incremental updates in shadow deployments.

4.3 LLMs to combat phishing

Phishing remains one of the most persistent socio-technical threats because it blends deception (language, branding, urgency cues) with rapidly changing delivery channels (email, web, QR/quishing, chat, and voice). LLMs are increasingly adopted in phishing defense not only as high-capacity text classifiers, but also as semantic reasoning layers that can fuse heterogeneous evidence (message content, URLs, DOM/HTML features, screenshots, and organizational context) and produce actionable explanations for analysts.

4.3.1 Detecting Phishing

a) Explainable LLM/Transformer detectors for phishing emails A recurring line of work treats phishing detection as a supervised discrimination task over email text and metadata, often strengthened with interpretability to support SOC adoption. Uddin et al. [124] follow this direction by combining a DistilBERT-based detector with post hoc explanations (e.g., LIME-style attribution) to improve transparency while retaining high accuracy. Such designs typically improve analyst trust and triage speed, but their reliability depends on (i) realistic evaluation against *rewritten* or *LLM-paraphrased* phishing and (ii) stability of explanations under distribution shift.

b) Prompted LLM detectors (“LLM-as-a-judge”) with structured evidence Several systems rely on instruction-tuned LLMs (e.g., GPT-class models) and convert raw emails/webpages into structured prompts to obtain both a decision and a rationale. Koide et al. [125] exemplify this approach for email phishing detection, emphasizing explainable outputs alongside strong reported accuracy. The main advantage is fast bootstrapping with minimal training data; the key limitation is *operational cost/latency* and vulnerability to *prompt manipulation* or context truncation when emails are long or contain obfuscated content.

c) Multimodal website phishing detection (brand, DOM, visuals) For phishing websites, LLMs/MLLMs are commonly used to combine textual cues (page content, URL tokens, DOM/HTML structure) with visual branding cues (screenshots, logos). Lee et al. [126] proposed a two-stage approach that first infers the claimed brand and then checks brand–domain consistency. Koide et al. [127] similarly leveraged LLM reasoning over crawled page context to identify impersonation and social engineering cues. Recent robustness-focused studies highlight that *adversarial perturbations* (visual/textual changes, DOM obfuscation, and cloaking) can degrade detectors and must be explicitly evaluated. Kulkarni et al. [128] provided an adversarial robustness analysis of phishing webpage detectors when moving from classical ML to LLM-based models, reinforcing the need for threat-model-aware evaluation.

d) Multi-agent LLM pipelines for email/web phishing A growing trend is to decompose detection into specialized roles (e.g., content analyzer, URL/WHOIS checker, brand verifier, policy checker), coordinated by an LLM-driven controller. MultiPhishGuard [129] illustrated this multi-agent paradigm for phishing email detection, where agents cross-validate signals to reduce single-point failures. This pattern tends to improve robustness and interpretability, but it increases system complexity, raising new requirements for audit logging, failure isolation, and safe tool access.

e) Efficiency and deployment constraints (quantized LLMs and hybrid embeddings) Because SOC pipelines often require near-real-time screening, many works adopt hybrid designs: LLM embeddings + lightweight classifiers, or quantized/small LLMs for edge/IoT gateways. Thapa et al. [130] compared quantized LLMs against classical models in the Gen-AI era, highlighting practical latency–accuracy trade-offs. Similarly, LLM-derived embeddings can be paired with conventional ML classifiers to reduce inference cost while

preserving semantic sensitivity [131]. In IoT contexts, integrated deep models augmented with LLM-based analysis have been explored to balance detection accuracy with constrained compute budgets [132].

f) Emerging channels (quishing, chat phishing, and voice phishing) The attack surface is expanding beyond email/web. Weinz et al. [133] empirically studied emerging threats such as quishing and LLM-generated phishing, underscoring that organizational exposure is driven by both technical bypass and human factors. For voice-based social engineering, LLM-based augmentation has been used to improve data coverage and classification performance in voice phishing detection [134]. These directions motivate detection pipelines that can ingest multimodal evidence and maintain consistent decision criteria across channels.

g) Offense-aware evaluation Some studies analyze how generative models can increase phishing effectiveness and, critically for defense, how detectors behave under *LLM-assisted paraphrasing* and *adaptive content*. Fairbanks et al. [135] highlighted that adaptive generation can reduce detectability and stress-test institutional defenses. Afane et al. [136] further discussed how LLM agents can amplify the scale and adaptability of phishing campaigns, reinforcing that detection benchmarks should include LLM-rewritten, multilingual, and context-conditioned variants.

h) Takeaways Across papers, the most consistently effective phishing detectors are those that (i) fuse *content semantics* with *external evidence* (URLs, domain reputation, brand consistency), (ii) incorporate *interpretability* for analyst validation, and (iii) evaluate robustness under *adversarial* and *LLM-generated* transformations. Conversely, failures often arise from unrealistic datasets, narrow threat models (e.g., excluding cloaking/adversarial edits), and deployments that ignore latency/cost constraints or prompt-level attack surfaces.

Table 5 synthesizes recurring architectural patterns for LLM-enabled phishing defense and contrasts their practical trade-offs, with emphasis on deployment latency and operational constraints.

4.3.2 Mitigating phishing

Phishing mitigation extends beyond flagging messages: it includes *reducing user susceptibility*, *blocking delivery/infrastructure*, and *hardening workflows* against LLM-enabled adaptation. We group the literature into (i) prevention via awareness/training, (ii) operational defenses against LLM-enabled phishing, and (iii) ecosystem-level measures.

a) Human-centric mitigation and adaptive training: LLM-enhanced Security Awareness Training (SAT) aims to personalize training content, simulate realistic scenarios, and provide feedback aligned with users' roles and observed weaknesses. Hafzullah [137] reported that LLM-driven SAT can significantly reduce phishing susceptibility in measured populations. This line of work is valuable when paired with organizational controls (reporting channels, response playbooks), but it must be evaluated longitudinally to measure retention and real-world transfer.

b) Defending against LLM-enabled phishing content: As phishing emails become more fluent and context-aware, mitigation increasingly targets the *generation–detection gap*. SoK-style analyses aim to systematize how LLM phishing is generated and why it bypasses detectors, and to identify countermeasures; see, e.g., Chen et al. [138]. Complementing analysis, new defense paradigms propose explicit mechanisms to make phishing triggers detectable or disrupt the attack pipeline, e.g., trigger-tagging defenses for LLM-enabled phishing emails [139].

Table 5 Synthesis of LLM-enabled phishing detection patterns: design choices, strengths/limits, and expected latency

Pattern	Typical instantiation	Pros/cons (operational)	Latency (expected)
Transformer classifier (email)	Fine-tuned encoder (e.g., DistilBERT/BERT) on email text + headers; optional XAI	+ High throughput, low cost; easy inline filtering. + Stable decision boundary if trained well. -- Needs frequent re-training for <i>LLM-paraphrased</i> phishing; may miss context beyond features. -- Explanations may be brittle under shift	Low (ms–tens of ms)
Prompted LLM (LLM-as-a-judge)	Structured prompt with email/webpage evidence; LLM outputs label + rationale	+ Strong semantic reasoning; high quality rationales for analysts. + Works with limited labeled data. -- Higher cost and variable latency; sensitive to prompt/context length. -- Requires guardrails against prompt injection/manipulation	Medium–High (hundreds ms–seconds)
Multimodal webpage analysis	Brand inference + domain consistency; DOM/HTML + screenshot cues (MLLM)	+ Captures visual impersonation + layout/DOM cues; strong against simple text obfuscation. -- Vulnerable to cloaking and adversarial edits unless explicitly evaluated. -- Higher compute; requires rendering/screenshot pipeline	Medium (rendering + inference)
Hybrid retrieval + evidence grounding	RAG over CTI, policies, known-brand corpora; LLM uses retrieved evidence for decision	+ Improves traceability and reduces stale knowledge. + Useful for organization-specific rules/policies. -- Retrieval adds latency and new attack surfaces (poisoned docs, instruction injection). -- Needs provenance + filtering for SOC use	Medium (retrieval + inference)
Multi-agent pipeline	Specialized agents (content/URL/WHOIS/brand/policy) with cross-checking and arbitration	+ Better robustness via redundancy; supports modular auditing. + Natural mapping to SOC triage roles. -- More engineering complexity; tool security and message governance required. -- Latency increases with the number of agents/tools	Medium–High (multi-step)
Efficiency-first (quantized/embeddings)	Quantized small LLMs, or LLM embeddings + lightweight classifier for gateways/IoT	+ Practical deployment under tight compute/latency budgets. + Good compromise for inline screening. -- Potential accuracy drop; needs careful calibration and drift monitoring. -- May underperform on highly contextual or targeted spear-phishing	Low–Medium (device-dependent)

c) Infrastructure- and ecosystem-level mitigation (domains, crawling, and cloaking): Traditional anti-phishing measures remain essential (domain takedown, reputation, blacklist/whitelist, URL filtering), but LLM-era threats intensify the need for rapid discovery and analysis of evasive sites. Abuadbbba et al. [140] showed the benefit of enriching phishing signals with external sources (e.g., reputation/registration signals) to detect subtle cases. Moura et al. [141] discussed ecosystem-level characterization and mitigation for ccTLD phishing. For cloaked websites that present different content to crawlers vs victims, LLM-driven adaptive crawling has been proposed to uncover hidden phishing infrastructure (e.g., PhishParrot) [142].

d) Takeaways: Mitigation is most effective when detection outputs are integrated into operational loops: (i) automatic quarantine/URL blocking for high-confidence cases, (ii) analyst-in-the-loop verification for borderline cases, and (iii) feedback into training and policy updates. LLM-based mitigation adds value primarily in *explanation*, *evidence correlation*, and *adaptive training*, while classical controls remain preferable for low-latency, high-throughput enforcement (e.g., URL filtering, reputation checks). Accordingly, Q1-grade evaluations should report not only accuracy but also robustness (LLM-paraphrase, adversarial edits, cloaking), operational cost, and end-to-end response time.

Table 6 synthesizes LLM-enabled *mitigation* patterns for phishing, focusing on how models are integrated into SOC and email/web workflows (beyond detection), the operational benefits, and the key risks that must be controlled.

4.3.3 Open research gaps

While many phishing studies report very high accuracy on curated datasets, several gaps remain for realistic deployment. First, *concept drift* (new sender infrastructure, evolving templates, multilingual and multimodal variants, seasonal campaigns) is rarely evaluated via time-split protocols or shadow deployments, so robustness over time remains unclear. Second, *adaptive attackers* can rapidly mutate content and evade lexical cues; benchmarks often under-represent these adaptive behaviors and do not stress-test models against adversarial template manipulation or distribution shift. Third, operational deployment raises *evidence and governance* questions (what headers/URL artifacts are retained, how user privacy is preserved, and how decisions are explained and audited within SOC workflows). Future work should prioritize drift-aware evaluation, adaptive adversary testing, and pipeline-level controls that enforce evidence-linked and auditable conclusions.

4.4 LLMs to combat SQL injection

SQL injection attacks obtain sensitive information without authorization by taking advantage of flaws in database query processing. LLMs provide efficient solutions by enhancing the detection of malicious queries and assisting with mitigation strategies. These aspects are examined in this section (Table 7). Importantly, the threat surface has expanded with the adoption of *LLM-integrated* applications (e.g., tool-using agents, RAG pipelines, and Text-to-SQL systems), where attackers may trigger SQLi indirectly through prompt-to-SQL behaviors or poisoned model states. Recent work shows that SQLi can be *migrated* into Text-to-SQL models via backdoor-style mechanisms, creating a new class of *model-level* SQLi risks beyond classical web input vectors [143].

Table 6 Cross-paper synthesis of LLM-enabled phishing mitigation patterns: where LLMs sit in the pipeline, what they automate, and the operational trade-offs

Mitigation pattern	Where it integrates	What it mitigates/automates	Pros/cons (SOC realism)	Latency impact
LLM-assisted user training (SAT)	Awareness platform; interactive coaching	Adaptive lessons, tailored feedback, scenario-based simulation, post-training assessment (PPP reduction).	+ Scales human factor mitigation; measurable outcomes. + Personalization improves engagement. – Does not stop attacks instantly; depends on adoption. – Risk of over-trust/"training fatigue" if poorly designed	Low (offline)
Policy-driven triage + auto-playbook drafting	SOC ticketing/IR; analyst copilot	Drafts containment steps, IOC extraction, communications templates, and remediation checklists from email evidence and org policies.	+ Faster triage; reduces analyst load and inconsistency. + Improves documentation quality. – Requires strict guardrails; hallucinated steps are risky. – Must be constrained to approved playbooks and audited outputs	Medium (seconds)
Inline content transformation/warning banners	Email gateway/client UI	Rewrites or annotates suspicious messages, adds safe previews, highlights risky intents/requests, and blocks risky actions (links/macros).	+ Prevents clicks at the point of decision. + Works even when detection confidence is moderate (human-in-the-loop). – UX trade-off; false alarms reduce trust. – Attackers may adapt content to evade warning heuristics	Low–Medium
Trigger-tag/provenance-based defenses	Email pipeline; model-side controls	Adds robust tags/markers to messages or model inputs to help downstream LLMs distinguish instructions vs data and resist prompt-based manipulation.	+ Addresses the <i>LLM-specific</i> risk of instruction hijacking. + Improves audibility of "why the model acted." – Needs ecosystem support (clients, gateways, or LLM middleware). – Standardization is still evolving; partial deployment limits benefits	Low
Domain takedown/web mitigation support	Brand protection + registrar coordination	Summarizes evidence; generates abuse reports, ranks takedown priority, and tracks remediation status across stakeholders.	+ Accelerates response and coordination. + Useful for ceTLD/registrar workflows and large-scale campaigns. – Does not replace legal/registrar processes. – Evidence must be verifiable; avoid fabricated claims	Low–Medium
Cloaking-aware discovery/adaptive crawling	Threat intel/phishing site hunting	Guides crawling strategy to expose cloaked phishing, expands campaign coverage, and supports early blocking.	+ Improves coverage against evasive infrastructure. + Useful upstream of filtering and takedown. – Higher engineering complexity; may raise ethical/legal constraints. – Needs careful rate limits and safe automation boundaries	Medium–High
Org-specific chat/voice phishing mitigation	Enterprise comms (chat/VoIP)	Assists with scripts, warnings, and classification in chat/voice channels where social engineering is evolving.	+ Extends mitigation beyond email. + Few-shot adaptation to business context is possible. – Privacy constraints; data retention must be controlled. – Higher false positive risk for informal language/accents	Low–Medium

Table 7 Synthesis for SQLi defenses: patterns, benefits/limits, and typical runtime posture

Pat.	Representative idea (examples)	Pros/Cons (why it helps or fails)	Latency
P1	Query/log classification [144, 145]	+ Fast scoring for high-volume streams; + easy deployment; – brittle under obfuscation/novelty if context is missing	Inline/near-inline
P2	Agentic black-box probing [146, 147]	+ Finds exploitable SQLi without source; + adapts payloads; – higher cost/latency; – requires safe testing constraints	Offline/triage
P3	LLM + static/taint to generate rules [148]	+ Improves coverage and precision via program structure; – depends on code availability; – toolchain integration effort	Offline (CI/CD)
P4	Adversarial SQLi generation [149]	+ Stress-tests Web Application Firewall (WAF)/filters; + reveals detection gaps; – dual-use risk; – must be sandboxed	Offline/red-team

Legend: P1–P4 correspond to architectural patterns described in the synthesis above

4.4.1 Detecting SQL injection

The SQLi detection literature can be organized along the four recurring patterns (P1–P4). Rather than treating these lines as competing, recent work shows they are often complementary in end-to-end pipelines (e.g., fast inline screening, followed by agentic probing or code-centric confirmation).

a) Query-centric classification: From hybrid IDS to Transformer-assisted cascades: Early learning-based defenses aimed to reduce the brittleness of rule/keyword matching by modeling *normal* query behavior and flagging deviations. Ali et al. [150] proposed a hybrid IDS that fuses misuse and anomaly detection: typical behavior is stored (XML) and user profiles are derived from benign query data via data mining, enabling a more systematic separation between legitimate and fraudulent requests. Empirically, the hybrid design improves detection rates while reducing false positives compared with conventional single-paradigm approaches, supporting practical deployment where alert fatigue is a primary constraint.

More recent designs incorporate modern NLP representations while explicitly targeting operational constraints. Taşdemir et al. [144] introduced a cascaded detector that couples transformer-based NLP encoders with traditional ML classifiers, achieving very high accuracy (reported at 99.86%) while substantially reducing inference time compared to Transformer-only baselines. This architecture is particularly aligned with high-throughput environments (e.g., data centers), where the central trade-off is *latency vs. accuracy*, i.e., a lightweight second-stage classifier can preserve detection quality while meeting strict processing budgets.

b) Agent-based black-box detection: LLM-guided probing and reasoning: A growing set of works targets realistic settings where source code and internal telemetry are unavailable. In this line, LLMs are not only used as classifiers but as *reasoners* that interpret SQLi cues, choose probing steps, and adapt payloads. SqliGPT [147] framed LLMs as black-box SQLi detectors and leveraged them to automate probing strategies, emphasizing the practicality of detection when the system under test is accessible only via its interface. Similarly, Zhang et

al. [145] proposed an LLM-based pipeline focused on malicious-query recognition as a dedicated detection workflow. Yang et al. [146] adopted multi-agent cooperation to emulate the sequential reasoning process of human security testers. Their LLMSQLi tool combines LLM context understanding with agentic division of labor to generate tailored test payloads and improve detection efficiency in black-box web-application testing. Reported results indicate successful detection across targets in SQLiMicroBenchmark and improved efficiency relative to established black-box scanners (e.g., SQLmap and Arachni), illustrating how *agentic probing* can complement traditional scanning heuristics when adaptive testing is required.

c) Code-centric detection and rule synthesis: LLMs as security engineering copilots: Beyond runtime detection, an emerging thread treats LLMs as automation engines for producing *project-specific* vulnerability checks. Hu et al. [148] proposed QLPro, integrating LLMs with static taint analysis to automatically generate syntactically correct and precise CodeQL rules at scale. By combining a triple-voting classifier with a structured multi-role mechanism, QLPro targets a key limitation of traditional static analyzers: reliance on generic rules and manual authoring effort. This code-centric framing shifts the objective from predicting whether a single query is malicious to improving *coverage and precision* of vulnerability discovery over large codebases.

d) Adversarial generation: Strengthening evaluation under adaptive attackers: Detection quality is increasingly stress-tested under adaptive payload generation rather than static signatures. Leung et al. [149] introduced XploitSQL, combining language models with reinforcement learning to generate stronger adversarial SQLi payloads. While not a detector per se, this line is central for rigorous benchmarking: models optimized on fixed corpora may overestimate robustness unless evaluated against *adaptive* and *model-assisted* adversaries.

e) LLM-integrated applications: Prompt-to-SQL injection risks and defense-aware detection: When SQL queries are produced indirectly by LLM-driven agents (e.g., tool-using chains), the attack surface expands from classic SQLi to prompt-mediated pathways. Pedro et al. [151] analyzed prompt-to-SQL (P2SQL) injection risks in LLM-integrated web applications (including LangChain-based designs), showing that prompt injection can induce malicious SQL generation and proposing multiple defense strategies to mitigate such attacks. A follow-up study [152] systematized these risks by clarifying attacker goals (e.g., data exfiltration and policy bypass) and organizing defenses into layers such as prompt isolation, tool mediation, and validation, which directly informs how detection should be placed in the pipeline (before generation, at tool boundaries, and post-generation validation).

f) Comparative evidence and system-level risk modeling: Recent comparative work highlighted that LLM-based approaches are not uniformly superior across operational criteria. Maligazhdarova et al. [153] contrasted ML and LLM-based methods for SQL/NoSQL injection detection, emphasizing practical trade-offs among robustness, explainability, and deployment cost. Complementarily, Tete et al. [154] examined broader security risks in LLM-enabled applications via STRIDE+DREAD threat modeling, explicitly including SQL injection among other classes (e.g., prompt injection and jailbreaking).

This system-level analysis reinforces that effective SQLi mitigation increasingly requires *multi-layered controls*: fast query-level filters (P1), black-box probing for unknown surfaces (P2), code-level rule generation for scalable assurance (P3), and adversarial testing to avoid brittle evaluations (P4).

4.4.2 Mitigating SQL injection

Mitigation in the LLM era typically appears in three layers: (L1) *classical query safety* (sanitization, query-structure constraints, runtime enforcement); (L2) *LLM-aware media-*

tion (tool gating, prompt/data separation, structured tool calls); and (L3) *adaptive perimeter defenses* where generative artificial intelligence (GenAI) is used to evolve WAF rules and harden filters against novel payloads. In addition, model-level defenses become necessary for Text-to-SQL systems where backdoor-style mechanisms can steer SQL generation toward malicious patterns [155, 156]. Table 8 summarizes these mitigation layers.

a) Classical query safety: Sanitization, structural constraints, and runtime enforcement: In [157], the authors positioned ASSIST as a compile-time transformation approach that automatically inserts context-sensitive sanitization into web applications. The paper contrasted ASSIST with widely deployed but fragile defenses such as PHP Magic Quotes, which blindly escape special characters (e.g., quotes) and often over-sanitizes while failing to reason about context (including cases where user input mixes SQL with non-SQL content such as HTML/JavaScript). In contrast, ASSIST leveraged the intended SQL query structure and database schema to place sanitizers only where needed, thereby improving precision and reducing unnecessary input distortion. The work also differentiated ASSIST from approaches such as AMNESIA, which combined static analysis with runtime monitoring to verify that issued queries match pre-learned valid templates. By injecting sanitization logic directly at the query-construction sites during compilation, ASSIST reduced dependence on runtime inspection and avoided the overhead commonly associated with dynamic taint tracking, while going beyond classic static analyzers that typically *report* vulnerabilities without automatically repairing them.

Rasoul Jahanshahi et al. [158] proposed SQLBLOCK, a hybrid static–dynamic mitigation mechanism tailored to legacy PHP applications, including object-oriented codebases. SQLBLOCK statically identifies database APIs and relevant procedures and then learns function-specific query descriptions during a training phase. These descriptions define allowed query patterns that are subsequently enforced at runtime to block unauthorized queries. Evaluations across vulnerable real-world plugins and applications (e.g., WordPress/Drupal ecosystems) reported low false-positive behavior and limited response-time overhead, suggesting that structural query constraints can be practical when the protection boundary is carefully scoped and trained per application function.

In a recent work, Kim and Lee [159] explored SQL injection fuzzing using LLMs to improve the generation and prioritization of test inputs, reflecting a trend where classical enforcement is complemented by stronger, model-assisted test discovery. Similarly, Dat et al. [160] combined LLM capabilities with traditional fuzzing to enhance vulnerability discovery in web applications, indicating that defenses increasingly benefit from more adaptive testing loops rather than static payload sets.

b) LLM-aware mediation: Securing tool-using and prompt-driven SQL pipelines: As LLMs are integrated into applications that generate or mediate SQL (e.g., via agents or chain-based orchestration), mitigation must address prompt-to-SQL escalation paths. Pedro et al. [151] analyzed how prompt injection can be leveraged to induce SQL injection behaviors in LLM-integrated web applications and discuss defense strategies aimed at reducing this risk in practice. The subsequent ICSE study [152] systematized attacker goals (e.g., data exfiltration and policy bypass) and organizes defenses into layered controls, including prompt isolation, tool mediation, and validation at execution boundaries, directly aligning mitigation with the points where LLM outputs become actionable SQL.

Chen et al. [117] addressed this problem class by proposing *structured queries* that separate instructions from data into distinct channels, to ensure the model follows only the instruction channel while treating the data channel as non-executable content. Their results indicated improved robustness to prompt injection with limited utility degradation, and the approach

Table 8 SQLi mitigation: synthesis of defense layers for classical and LLM-integrated settings

Layer	Mechanism	Representative works	Notes (pros/cons)
L1: Classical query safety	Context-aware sanitization; enforce allowed query structures at runtime	[157, 158]	Strong baseline; low overhead; may miss prompt-mediated SQLi in agent/tool pipelines
L2: LLM-aware mediation	Separate instructions from data; tool-call constraints; robust prompting/policies	[117, 152]	Targets prompt-to-SQL pathways; requires disciplined tool/API integration
L3: Adaptive perimeter defenses	Auto-evolve WAF rules and filters using GenAI	[155]	Better coverage for emerging payloads; needs careful validation to avoid over-blocking
Model-level (Text-to-SQL)	Backdoor-aware training/verification; dataset and pipeline hardening	[156]	Addresses risks unique to Text-to-SQL; still emerging and less standardized

is conceptually compatible with tool-gated architectures where SQL execution is mediated through structured interfaces.

Mitigation can also be improved upstream during code generation and developer workflows. For instance, Jamdade et al. [161] studied how prompt refinement patterns can reduce the likelihood that LLM-generated web code contains SQLi (and other common) vulnerabilities, implying that “secure-by-prompt” practices can serve as an additional, lightweight control, though they should be treated as supportive rather than sufficient in isolation. At the model interaction layer, jailbreak-style techniques highlight why controls must be explicit. Indeed, Zhao et al. [162] introduced SQL Injection Jailbreak (SIJ), showing that injected jailbreak content in prompts can steer models toward producing malicious outputs. While the paper proposes a simple defense (Self-Reminder-Key), its broader implication is that tool-using systems should rely on external, enforceable mediators (e.g., structured tool calls, validators, and least-privilege execution) rather than assuming the model will reliably self-regulate.

c) Adaptive perimeter defenses: Evolving WAF rules and filters with GenAI: Beyond application-internal controls, perimeter hardening increasingly leverages generative methods to keep pace with payload mutation. Babaey and Ravindran proposed GenSQLi [155], a generative workflow that synthesizes and validates new defensive rules for web application firewalls, aiming to continuously update protections as adversarial SQLi patterns evolve. This “defense evolution” framing is especially relevant in high-traffic deployments where filters must adapt without frequent manual rule engineering.

Related hybrid detection-and-mitigation pipelines also appeared in the literature, combining text mining with generative models to improve the identification of SQLi patterns under distribution shift [163]. Although often presented as detection, these systems can be integrated at the perimeter as adaptive filters when paired with robust validation and monitoring.

Finally, LLM-era mitigation must account for cases where SQL is generated by Text-to-SQL models. For instance, Lin et al. [156] demonstrated that Text-to-SQL systems can inherit persistent, SQLi-like malicious behaviors via backdoor mechanisms, motivating model-level controls such as training-data hygiene, backdoor testing, and verification/validation of generated SQL before execution. This expanded the mitigation scope beyond query sanitation and mediation to include assurance activities within the model development and deployment pipeline.

d) Comparative evidence and practical trade-offs: Comparative studies reinforce that mitigation choices depend on deployment constraints. Maligazhdarova et al. [153] highlighted trade-offs between ML and LLM-based approaches in terms of robustness, interpretability, and operational cost, which map naturally onto L1–L3 mitigation layers. Specifically, lightweight structural constraints may be preferable for inline enforcement, whereas LLM-mediated controls are better suited for tool-using workflows, where context understanding and policy enforcement are required.

4.5 LLMs to combat malware

LLMs have proven to be a powerful tool in the fight against malware attacks. By leveraging their advanced natural language processing capabilities, LLMs can assist in detecting and mitigating malware. These aspects are examined in this section (Table 9).

Table 9 Synthesis for malware **detection**: architectural patterns vs. pros/cons vs. latency

Architectural pattern	Representative work(s)	Pros/Cons (typical)	Latency
Code/supply chain analysis (LLM over code)	SocketAI (npm) [164]	+ Strong semantic code cues; + catches obfuscation/typosquatting; – may be costly without prefiltering; – depends on prompt/coverage	Med
Long-context behavioral sequences	Syscall LLM + transfer [165]	+ Captures long-range dependencies; + good for subtle behaviors; – context window + compute cost; – needs careful windowing	Med–High
LLM reasoning with context extraction (Android-scale code)	LAMD [166]	+ Scales via key-context isolation + tier-wise reasoning; + adds explanation + consistency checks; – pipeline complexity; – still limited by extracted context quality	Med
Traffic/anomaly understanding with transformers	IoT malware anomaly analysis [167]	+ Good generalization across attack types; + contextual modeling; – sensitive to dataset shift; – feature representation matters	Low–Med
Adversarial malware generation (evaluation stress-test)	GLEAM [168]	+ Reveals detector weaknesses; + produces evasive variants; – mainly offensive/eval; – must be paired with defenses	(Offline)

4.5.1 Detecting malware

a) Code and supply chain understanding Zahan et al. [164] evaluated *SocketAI*, a workflow leveraging GPT-3/GPT-4 to detect malicious npm packages, and compared it against static analysis with CodeQL. *SocketAI* yields higher precision (GPT-4: 99%, GPT-3: 91%, CodeQL: 75%) and F1 (GPT-4: 97%, GPT-3: 94%, CodeQL: 85%), while recall remains comparable (95–97%). A hybrid design that used CodeQL as a pre-filter reduced the number of files sent to the LLM by 77.9% and lowered costs by 60.9%, improving scalability for practical deployment. Beyond metrics, the study highlighted LLM advantages in capturing diverse supply chain threats (e.g., data exfiltration, hidden backdoors, dependency obfuscation, and typosquatting) that may evade rule-based static checks.

b) Long-context behavioral modeling Sanchez et al. [165] proposed an LLM + transfer-learning framework for malware detection from system-call sequences. By fine-tuning long-context models (e.g., BigBird/Longformer) on a large-scale system-call corpus, the approach better captured extended dependencies than traditional n-gram pipelines and avoids the limitations of recurrent models on long sequences. Reported results showed strong accuracy (up to 0.86) and competitive F1 scores, with larger context windows improving detection of complex malware families (e.g., Bdlv and RansomwarePoC). While precision can lag kernel-level methods, the method targeted real-time operation via short (one-second) analysis windows and tunable decision thresholds to balance sensitivity and computational budget.

c) Adversarial generation and detector robustness evaluation Devadiga et al. [168] introduced *GLEAM*, combining GANs with LLM embeddings to generate evasive adversarial malware samples (hex/opcode features) that evade black-box ML detectors. Their results showed an average evasion-rate increase of 22.6%, underscoring that LLM-assisted pipelines should be evaluated not only on clean test sets but also under adversarial stress conditions.

d) Network/IoT malware via traffic anomaly understanding Omar et al. [167] studied LLM-family models (notably BERT and GPT-2) for anomaly detection in network traffic targeting IoT malware. They reported that BERT consistently outperformed GPT-2 across accuracy, precision, recall, and F1, suggesting that contextual representation learning can improve generalization to unseen/zero-day-like patterns compared with signature-based and standard ML/DL baselines.

e) Context extraction and tiered reasoning for large programs Recent Android-focused systems strengthened the idea of extracting security-critical evidence before reasoning. LAMD [166] emphasized contextualization over app evidence to reduce hallucinations and improve explainability, effectively using the LLM as a reasoning layer over curated features rather than raw code at scale. In a similar spirit, AppPoet employed a multi-view prompting strategy (e.g., permissions, APIs, components, metadata) to elicit structured judgments and analyst-readable reports for Android malware detection [169]. Few-shot and multi-label regimes were also explored. For instance, LeoDroid studied LLM-based few-shot multi-label detection to reduce labeled-data requirements while supporting multiple labels per sample [170].

f) Pipeline-level robustness and adversarial manipulation Beyond sample-level evasion, robustness is increasingly treated as a first-class concern. Shafee et al. show that LLM-based CTI/text-centric security pipelines can be degraded by evasion/flooding/poisoning-style inputs, which can cascade into worse downstream triage decisions [171]. Complementarily, Lan and Nait-Abdesselam analyze LLM-driven feature-level adversarial attacks against Android malware detectors, arguing that evaluation should include adversarial stress testing and guardrails against spurious rationales [172].

4.5.2 Open research gaps

Before LLM-based pipelines, Android malware detection has been extensively studied using non-LLM ML/DL baselines trained on static and dynamic behavioral features, with widely used benchmarks such as *Drebin* (feature-rich static APK indicators) and *TUANDROMD* (dynamic behavior traces) serving as common evaluation grounds. Across this literature, CNN/RNN-style sequence models, graph-based variants, and autoencoder/anomaly detection approaches have demonstrated strong detection performance and *efficient inference* once feature pipelines are fixed, making them practical for large-scale mobile security deployment. However, these baselines remain sensitive to feature drift (API/permission evolution), dataset bias, and adversarial adaptation, motivating the more recent interest in LLM-enabled systems that can integrate heterogeneous evidence (permissions, logs, reports, tickets) and provide analyst-facing semantic explanations while introducing new risks and cost/latency constraints.

4.5.3 Mitigating malware

Mitigation malware approaches discussed within this subsection follow three mitigation patterns: (i) uncertain execution + perturbation, (ii) moving target defense at the system level, and (iii) LLM-guided triage as highlighted in Table 10.

a) Uncertain execution and perturbation In [173] Ruimin Sun et al. proposed a novel approach to the urgent problems of real-time malware detection, which establishes two different execution environments: an uncertain environment for software that receives borderline classification and a standard environment for benign software. By using random perturbations that prevent potentially harmful software from running while having the least possible effect on innocent applications, CHAMELEON aims to improve malware detection capabilities. Their study's findings demonstrate the framework's efficacy, as 92% of malware had execution disruptions compared to just 10% of benign software, confirming its capacity to target criminal activity while preserving the functionality of legitimate applications. Furthermore, CHAMELEON has a dynamic perturbation threshold that modifies the degree of disruption according to program activity, which significantly lowers the number of successful malware executions. CHAMELEON provides a supplementary strategy that uses uncertainty to improve detection efficacy in contrast to conventional antivirus technologies, which primarily rely on signature-based detection and frequently struggle with novel and evolving threats, as well as behavior-based approaches that can lead to high false-positive rates.

b) Moving target defense To combat ransomware attacks, the authors of [174] introduce MTFS (Moving Target Defense-Enabled File System), a revolutionary method of malware mitigation. It presents three cutting-edge Moving Target Defense (MTD) strategies: concealing file types, trapping recursive directory traversal, and delaying attackers. By integrating directly into the file system, these methods are intended to proactively protect against ransomware, increasing the efficacy of malware mitigation on devices with limited resources, including Internet of Things systems. By changing the attack surface, MTFS aims to develop a dynamic defense mechanism that makes it harder for attackers to succeed and less predictable. According to experimental data, MTFS is effective; it successfully operated on a Raspberry Pi device and was able to combat two ransomware samples. In a more extensive testbed scenario, MTFS faced 13 ransomware samples and preserved 97% of the targeted data while using minimal resources, demonstrating its viability for practical implementation. While many of the current solutions rely on network-based techniques, the report also points

out the shortcomings of current malware defenses, which frequently concentrate on data restoration or detection rather than prevention.

c) LLM-guided triage: LLMs are also increasingly used *after* detection to accelerate mitigation workflows such as triage, tactic/technique labeling, analyst summarization, and response planning. Malgta, for example, investigates LLM-based guided malware tactical analysis, aiming to convert raw indicators and behavioral evidence into structured, action-oriented analysis that can support incident response and containment decisions [175].

d) Comparative evidence While CHAMELEON and MTFS illustrate *system-level* defenses that do not depend on perfect classification, LLMs are increasingly relevant for mitigation as an *operator-facing accelerator* (triage, explanation, playbook guidance) and as a component whose robustness must be explicitly protected. Therefore, a defensible malware mitigation stack increasingly looks like: (i) stable prevention controls (e.g., MTD/containment), (ii) detection with prefiltering + constrained LLM reasoning, and (iii) verification/guardrails to prevent adversarial content from corrupting CTI and response pipelines [171].

4.5.4 Takeaways

Across recent studies, LLM-based malware defense tends to follow a set of recurring design patterns: (i) *code/supply chain understanding* (e.g., detecting malicious packages or suspicious code regions), (ii) *long-context behavioral modeling* (e.g., system-call or execution-trace sequences), (iii) *context extraction + tiered reasoning* to fit large programs into limited context windows, and (iv) *LLM-assisted CTI/triage* (summarization, tactical interpretation) as a human-in-the-loop accelerator. These approaches typically succeed when they (a) isolate security-critical context before reasoning (to avoid drowning the model), (b) enforce factual-consistency checks or evidence grounding to reduce hallucinations, and (c) use LLMs as a *reasoning layer* on top of robust prefilters (static analysis, embeddings, heuristics) to control latency and cost. They often fail when prompts expose overly broad context (low signal-to-noise), when adversarial inputs exploit model brittleness (prompt-level or feature-level manipulation), or when the pipeline lacks guardrails against spurious rationales and overconfident explanations.

4.5.5 Open research gaps

Malware ecosystems evolve rapidly (packing/obfuscation, new families, supply chain vectors), yet many studies rely on static, offline datasets that may not reflect current attacker tactics. Key gaps include robustness to *obfuscation and polymorphism*, cross-domain generalization across device/OS versions, and realistic evaluation against *adaptive evasion*. For LLM-augmented pipelines, additional risks include prompt-based manipulation of reports and retrieval poisoning when using external CTI. Future work should emphasize time-resolved datasets, adversarial robustness tests, and evidence-grounded workflows suitable for SOC integration.

4.6 LLMs to combat XSS

By exploiting flaws in user input, XSS attacks introduce malicious scripts. LLMs are used to identify these attacks and mitigate their effects by enhancing application security. This part explains their role in detection and mitigation (Table 11).

Table 10 Synthesis for malware **mitigation**: patterns vs. pros/cons vs. latency

Mitigation pattern	Representative work(s)	Pros/Cons (typical)	Latency
Uncertain execution + perturbation	CHAMELEON [173]	+ Disrupts malware execution; + low overhead; – may affect borderline benign apps; – needs tuning	Low
Moving Target Defense at filesystem level	MTFS [174]	+ Proactive ransomware hardening; + fits resource-limited devices; – narrower scope (FS-centric); – attacker adaptation possible	Low
LLM-guided triage/CTI hardening (pipeline robustness)	CTI pipeline attacks [175]	+ Highlights where to add guardrails/verification; + improves resilience of downstream workflows; – not a standalone malware blocker; – requires pipeline redesign	Med

4.6.1 Detecting XSS

Research on XSS detection has progressively shifted from context-insensitive pattern matching toward *context-aware* analysis, reflecting the fact that exploitability depends on where and how attacker-controlled data is interpreted (e.g., HTML body vs. attribute vs. JavaScript vs. DOM-driven execution).

a) Strong ML baselines for payload detection and mitigation: Kshetri et al. [176] proposed *algoXSSF*, a ML-based framework designed to detect and mitigate both XSS and XSRF/CSRF in web applications. Compared with rule-based filters and static input validation, the framework emphasized adaptability and automation, reporting high detection performance (e.g., precision above 99% for several learners such as Random Forest, SVM, and LSTM-Attention). In addition to classification, the pipeline coupled detection with standard mitigation mechanisms (e.g., input validation, anti-CSRF tokens, and Same-Origin Policy constraints), supporting near real-time monitoring and response.

b) Context-sensitive analysis for framework-specific vulnerabilities: Although many detectors fail to account for the *execution context* in which sanitization occurs, Steinhäuser et al. [177] introduced *DjangoChecker* to identify context-sensitive XSS issues in Django-based applications. The tool combines server-side parsing with dynamic taint tracking to verify that sanitization matches the required browser context, addressing gaps in prior Python-centric tools (e.g., PyT, Taint Mode) that can miss vulnerabilities due to incorrect sanitization under nested contexts (e.g., JavaScript embedded in HTML). Their evaluation on real-world Django applications showed that DjangoChecker can uncover serious vulnerabilities that are missed by both commercial black-box scanners and context-insensitive static checks.

c) Obfuscation and domain shift: Although high-accuracy ML baselines are promising, cross-paper evidence indicates that *accuracy alone* is insufficient under heavy obfuscation and deployment-time domain shift (new frameworks, templating patterns, or previously unseen contexts). Recent studies, therefore, explore LLMs to (i) improve generalization via richer semantic representations and (ii) produce analyst-facing explanations that support trust and triage.

d) LLM-enhanced XSS detection trends: Miczek et al. [178] demonstrated how LLM-based reasoning can complement ML detectors, reducing brittleness to lexical variation

Table 11 Mini synthesis for XSS: architectural patterns vs. strengths/limits and deployment latency

Pattern	Pros/When it works	Cons/Latency
LLM-assisted classification (inline IDS/WAF)	Handles obfuscation better than purely lexical rules; can generalize across payload styles	Higher inference cost; needs guardrails for stability
Context/semantic modeling (DOM-/sink-aware)	Reduces context-insensitive FPs; stronger on DOM-based XSS and nested contexts	Requires richer traces (DOM snapshots, templates); feature extraction overhead
Generative defense (rule/sanitizer synthesis)	Scales rule authoring and produces human-readable patches/rules	Risk of over/under-sanitization; must validate outputs; often best offline

and improving robustness to payload diversity. Adiputra et al. [179] further investigated fine-tuning transformer-based LLMs for web vulnerability detection (including XSS), highlighting the practical direction of *domain-adapted* models. Tanaka and Mimura [180] similarly supported the feasibility of LLM-based XSS detection when prompts incorporate sufficient execution context to reason about injection intent rather than relying only on string signatures.

4.6.2 Mitigating XSS

A key synthesis insight is that mitigation approaches split into (i) *signature/policy enforcement* (fast, deterministic, but weaker on novel/semantic variants), and (ii) *semantic mitigation* (stronger generalization but higher cost and the need for validation). Accordingly, the study [181] focused on the ongoing threat of XSS vulnerabilities on the web, which still impacts several websites. To solve this problem, the authors provided XSnares, a cutting-edge client-side solution that uses a database of exploit signatures sourced from Common Vulnerabilities and Exposures (CVEs) to shield users against XSS exploits. XSnares's main goal is to offer application-specific security that successfully detects and prevents XSS attacks without the need for developer assistance or server-side fixes. In comparison to more general anti-XSS techniques that frequently fall short in thwarting particular attacks, the evaluation's findings suggested that XSnares effectively guards against 94.2% of the tested XSS exploits. Additionally, they explained how XSnares differs from other methods in that it uses a unique way to isolate vulnerable injection locations and apply client-side updates, which underlines how useful it is for improving user web security.

Also, Engin Krida et al. [182] discussed how to prevent XSS attacks, which are a prevalent flaw in online applications that hackers employ to steal private data like cookies and session IDs. In order to safeguard consumers without depending on service providers to address vulnerabilities, they suggested Noxes, the first client-side personal web firewall. Noxes is a web proxy that uses temporary, automated, and manual rules to filter HTTP requests. It also injects JavaScript to keep an eye out for dangerous behaviors like pop-ups and cross-domain frames. The program provides a workable substitute for server-side solutions by enabling users to secure their environments on their own. According to experimental assessments, it effectively blocked real-world XSS attacks, such as those that target PHP-Nuke and MYBloggie, while preserving usability by lowering the number of pointless connection notifications to 5.7%, guaranteeing 94.3% flawless browsing.

Recent work increasingly explored using LLMs to decide whether a stored payload is semantically malicious and to recommend context-specific neutralization strategies. For example, XSSshield [183] proposed LLM-based semantic understanding to defend against stored XSS, aiming to reduce bypasses that evade lexical filters and to improve decision justification for analysts. Similarly, Babaey and Ravindran proposed GenXSS [184], an AI-driven framework that automates XSS detection for WAF settings, aligning with a broader trend where LLMs help generate, refine, or explain filtering logic. In practice, such generative mitigations are often most reliable in an *offline* loop (rule proposal → validation → deployment), because direct inline rule generation can increase operational risk if outputs are not checked against application context and regression tests.

a) Comparative evidence: Client-side defenses (e.g., XSnares, Noxes) offer low-latency enforcement but are limited by signature coverage and deployment constraints. LLM-based semantic mitigation (e.g., XSSshield) can improve resilience to obfuscation and novel variants, but typically benefits from careful integration (caching, confidence thresholds, and safe fallback policies) to balance latency and safety.

4.6.3 Takeaways:

Across the literature, LLM-enabled XSS defenses converge toward three recurring architectural patterns: (i) *context-aware representation* of web inputs (URL/query/headers, HTML/JS snippets, DOM context) to reduce context-insensitive false alarms; (ii) *semantic reasoning* over rendered or reconstructed execution contexts (e.g., script intent, sink/source relationships, DOM-based flows); and (iii) *policy/rule generation* (WAF rules, sanitization templates, client-side guards) where LLMs act as generators and explainers rather than only classifiers. These patterns directly address classic XSS pain points, context sensitivity (HTML vs JS vs attribute), obfuscation, and domain shift, while introducing new trade-offs in latency, determinism, and robustness to adversarial prompt/input crafting.

4.6.4 Open research gaps

For web-attack detection, reported gains often depend on specific log sources, templates, and application contexts, which limits transferability across stacks and real production traffic. Key gaps include evaluation under *mixed benign noise*, encrypted/partial telemetry, and continuous drift as applications change. Moreover, when LLMs are integrated with developer tools (code review, WAF rules, SIEM queries), *prompt injection and unsafe tool outputs* become first-class risks. Future work should adopt realistic deployment protocols (shadow mode), enforce strict tool constraints, and evaluate robustness under adversarial prompt and payload manipulation.

4.7 LLMs to combat eavesdropping

LLMs improve network security by detecting and mitigating eavesdropping attempts through real-time anomaly detection and active response mechanisms. In modern systems, eavesdropping is no longer limited to classic RF interception: it also includes (i) *semantic eavesdropping* enabled by AI-assisted speech/text reconstruction from weak side-channels, and (ii) *data-leakage eavesdropping* in LLM-powered multi-agent ecosystems where sensitive content can be exposed through message flows. Consequently, LLMs are used both as (a) reasoning engines for interpreting heterogeneous signals (packets, logs, acoustic/PHY metadata) and (b) policy enforcers that govern what information can be transmitted, stored, or revealed.

4.7.1 Detecting eavesdropping

In [185] the authors focused on eavesdropping detection in the BB84 quantum key distribution (QKD) protocol, aiming to analyze and improve eavesdropping detection by distinguishing quantum channel errors. This study proposed a quantum bit error rate (QBER) comparison algorithm that uses the offending inequality to analyze the upper bound of the false positive and false negative ratios in eavesdropping detection, and a grouped BB84 protocol that maintains a polarization basis for a group of qubits, and introduces the combined QBER and group QBER criteria for accurate interception evaluation. Extensive simulations have shown that the QBER comparison algorithm can effectively detect eavesdropping attacks with high accuracy, while the algorithm-optimized grouped BB84 protocol guarantees at least 99.92% accuracy in detecting eavesdropping attacks under rapidly changing quantum channel conditions. They compared the proposed method with the existing technology and emphasized that the QBER comparison algorithm provides a trade-off between eavesdropping detection

Table 12 Synthesis of common architectural patterns for LLM-assisted eavesdropping defense (detection & mitigation)

Threat surface	LLM role	Observable signals	Pros	Cons/latency notes
Quantum channel interception (QKD)	Rule-based reasoning + statistical checks	QBER/group-QBER statistics	Strong theoretical grounding; low compute	Limited to QKD setting; relies on accurate channel modeling
LLM-agent message leakage	Policy enforcement/privacy guardrails	Agent-to-agent messages, tool I/O traces	Explicit control of sensitive flows; developer-friendly	Added runtime mediation; needs robust policy design
Side-channel & semantic eavesdropping (acoustic/ASR)	Context-aware semantic validation + anomaly reasoning	ASR transcripts, device/PHY metadata, sensor anomalies	Detects “meaning-level” leakage, not only signal-level anomalies	Detection can be noisy; must minimize false alarms to preserve UX
Wireless interception in dynamic LAWNs	RL planning with LLM-enhanced state representations	Routing/traffic features, PHY security indicators	Adaptive to topology changes; proactive defense	RL convergence overhead; online decisions must meet tight latency

accuracy and quantum resource saving, while the grouped BB84 protocol and the combined algorithm provide improved security performance compared to the classical BB84 protocol, especially under changing quantum channel conditions.

Beyond quantum settings, recent work shows that “semantic eavesdropping” can be amplified by LLMs, i.e., attackers may reconstruct *meaningful* content from weak signals and then rely on LLMs to correct/complete corrupted transcripts. EchoLLM [186] exemplified this trend by studying LLM-augmented reconstruction of spoken content from side-channel observations, which motivated the need for detection mechanisms that monitor not only raw signals but also suspicious downstream semantic artifacts (e.g., abnormal ASR confidence patterns, repeated near-duplicate recovered phrases, or unexpected context completions).

Furthermore, the authors Jian Cui et al. [187] introduced Maris, a privacy-enhanced development paradigm for multi-agent systems powered by LLMs, designed to prevent sensitive data leakage during agent-to-agent and agent-environment interactions. Existing frameworks like AutoGen and LangGraph lacked fine-grained control over message flows, making them vulnerable to attacks such as eavesdropping and prompt injection. Maris addressed this by embedding reference monitors and a quarantined local LLM that dynamically enforces privacy policies defined in a Data Protection Manifest. This integration allowed developers to safeguard communications without altering application logic. Evaluations across real-world tasks showed that Maris eliminated data leakage while preserving system performance and response quality.

Overall, the detection literature converges on two complementary directions: (i) *signal/statistics-first* detection (e.g., QBER-style criteria, traffic anomalies) and (ii) *semantics-first* detection, where LLMs interpret whether extracted content is plausible, policy-compliant, and consistent with expected context, helping surface stealthy interception that evades purely statistical thresholds.

4.7.2 Mitigating eavesdropping

In [188], the authors identified and categorized many types of jamming attacks, including frequency hopping, barrage, single-tone, multi-tone, intelligent reflective surface (IRS)-based, and AI-powered jamming. It examined previous studies on these risks, stressing the advantages and disadvantages of various strategies. According to the findings, smart jamming strategies like IRS-based and AI-powered attacks presented a greater challenge because of their versatility and sophistication, whereas traditional jamming techniques like barrage jamming are less successful at higher frequencies (such as THz). Despite potential trade-offs in data rate or implementation complexity, experimental evaluations demonstrated that secret communication techniques that make use of IRS-based systems or molecular absorption can successfully increase security by lowering the possibility of interception. Deep reinforcement learning and other AI-based anti-jamming techniques showed encouraging results in maximizing secrecy rates and thwarting dynamic jamming attempts.

Furthermore, Sathishkumar Nagarajan et al. [189] explored the possible misuse of AI-powered language models, such as ChatGPT and Gemini, in producing cyberattacks, such as eavesdropping, DoS, and phishing. Analyzing the features, limitations, and moral ramifications of employing AI models to create cyberattacks is the main goal, along with outlining the dangers and suggesting countermeasures to avoid misuse. They demonstrated that although AI models can be misused to produce complex cyberattacks, this misuse can be considerably reduced with the help of preventative actions and strong security procedures. The significance of ongoing security system monitoring and ethical issues is emphasized in the paper. It high-

lighted that because AI-generated attacks may replicate human behavior and provide realistic content, they are more complex and challenging to identify than previous approaches.

In [190], the authors addressed the security challenges in Low-Altitude Wireless Networks (LAWNs), especially the threat of eavesdropping. Traditional AI methods struggled to adapt in dynamic environments and lacked proactive defense. To overcome this, the authors proposed a novel framework using LLMs to enhance state representation and design intrinsic rewards for reinforcement learning. This approach enabled LAWNs to detect interception risks, adapt routing policies, and embed physical-layer authentication, significantly improving secure communication performance.

At the user-privacy layer, “Whispering Under the Eaves” [191] highlighted defenses against commercial and LLM-powered speech recognition that may inadvertently expose private speech. This line of work motivated mitigation strategies that combine: (i) privacy-preserving speech processing (e.g., controlled disclosure/redaction of sensitive entities), (ii) policy-aware on-device inference when feasible, and (iii) continuous monitoring to detect when downstream ASR behavior indicates an interception risk.

Finally, multi-agent LLM systems require explicit mitigation controls that treat *messages as an attack surface*, i.e., enforcing least-privilege tool access, minimizing sensitive context in prompts, and inserting runtime “privacy monitors” (as in [187]) to block unsafe disclosures before they leave the trust boundary.

4.7.3 Comparative evidence

To consolidate the diverse eavesdropping-defense literature, Table 12 provides a cross-paper synthesis that links each major threat surface to the *operational role* played by LLMs (e.g., rule-based reasoning, policy mediation, or RL-enhanced planning) and to the *signals* typically available for monitoring (from QBER statistics to agent/tool traces and PHY metadata). The table also highlights deployment-facing trade-offs, especially latency constraints and false-alarm sensitivity, which often determine whether a method is suitable for inline protection or better used for offline triage and auditing.

4.8 LLMs to combat MITM

Man-in-the-Middle attacks intercept communications between two parties to steal, tamper with, or manipulate sensitive data. LLMs can enhance detection and mitigation by analyzing traffic patterns and identifying anomalies in encrypted or untrusted channels. Importantly, “MITM” now spans two connected layers in modern systems: (i) *network/system MITM* (e.g., TLS interception, MQTT tampering, Open RAN/6G control-plane manipulation), and (ii) *LLM-mediated MITM* where adversaries tamper with *messages, tools, or interfaces* inside agentic LLM pipelines (e.g., multi-agent message manipulation, tool-metadata, GUI fingerprint injections). This section synthesizes both layers and clarifies where LLMs add value beyond classical IDS/ML.

4.8.1 Detecting MITM

In [192], the authors tackled the critical issue of cryptographic API misuse (CAM), which undermines the security of modern software systems. Traditional detection tools relied on static, expert-defined rules that fail to keep pace with evolving threats like MITM attacks. To address this, the authors introduced CryMisTa, a tool that leveraged LLMs such as GPT-4,

Table 13 MITM-oriented LLM architectures: patterns vs. pros/cons and latency implications

Pattern	Representative idea/integration	Pros/Cons	Latency
Code/Config auditor	LLM identifies crypto/API misuse enabling MITM (e.g., hostname verification disabled, weak cert handling) [192]	+ High coverage of diverse misuse patterns; + easy taxonomy expansion; – depends on code/context quality; – hallucination risk if unchecked	Offline/batch
Log/traffic semantic classifier	LLM fine-tuned/prompted for attack-type classification + mitigation hints from IoT logs [193]	+ Handles unstructured logs; + produces human-readable rationale; – model drift + prompt sensitivity; – may require guardrails	Nearline/online
Agent-message integrity monitoring	Detect stealthy multi-round message tampering in LLM-MAS pipelines [194]	+ Targets <i>intra-agent</i> MITM; + focuses on communication layer; – attacker can adapt; – needs protocol-level defenses	Online
Tool/metadata hardening	Adversary induces malicious tool invocation via metadata manipulation; motivates signed tool schemas and provenance checks [195]	+ Highlights tool-chain threat surface; + actionable controls (schemas, signatures); – requires ecosystem support; – tool trust remains central	Online
Closed-loop mitigation agent	LLM-guided mitigation loop in network control stacks (e.g., Open RAN/6G) (agentic detect → decide → act) [196]	+ Fast adaptation, policy-based actions; – risk of overreaction; – needs safe action constraints	Online (strict)

Claude, Gemini, and Llama to automatically detect and classify CAMs in real-world code. Through a three-phase process, identification, classification, and taxonomy construction, CryMisTa builds a dynamic and comprehensive taxonomy of misuse patterns. Notably, it identifies vulnerabilities directly linked to MITM attacks, such as disabling hostname verification and poor certificate management. By integrating newly discovered misuse types into existing tools, CryMisTa significantly enhanced its ability to detect and prevent cryptographic flaws, including those that enable MITM exploitation.

Furthermore, Jorge et al. [193] aimed to enhance threat detection and mitigation in IoT security logs using LLMs. The core problem addressed is the difficulty of analyzing unstructured, evolving log data, which traditional ML models struggle to handle effectively. To overcome this, the authors evaluated three open-source LLMs, DeepSeek-R1, LLaMA 3.2, and Qwen 2.5, using zero-shot, few-shot, and fine-tuning strategies. The proposed solution involved a pipeline that fine-tuned LLMs for multi-class attack classification and integrated IoT-specific mitigation recommendations. The Edge-IIoTset dataset was used to train and test the models across 14 attack types, including MITM. DeepSeek emerged as the most effective model, achieving high accuracy and generating context-aware mitigations. This dual capability of detection and response marks a significant advancement in automated IoT cybersecurity.

Beyond “classical” network MITM, recent works showed that LLM pipelines themselves can become MITM targets. For multi-agent systems (LLM-MAS), attackers can tamper with agent-to-agent messages over multiple rounds to steer decisions while remaining stealthy, motivating detection that monitors message integrity, consistency, and cross-agent agreement [194]. Complementarily, “tool-layer MITM” can induce unsafe tool calls by manipulating tool metadata (name/description/schema), highlighting the need for provenance-aware tool registries and signed schemas even when the user prompt is benign [195].

4.8.2 Mitigating MITM

CyberLLMInstruct [197] introduced a pseudo-malicious dataset that equips LLMs with detailed knowledge of cyber threats, including man-in-the-middle (MITM) attacks, which represent 10% of the dataset. By fine-tuning models like Llama 3.1 8B and Gemma 2 9B on this data, the paper demonstrates how LLMs can be trained to recognize, simulate, and respond to MITM scenarios. Although not a direct defense mechanism, this approach enables LLMs to act as intelligent assistants in identifying vulnerabilities and guiding mitigation strategies. The models are evaluated using the OWASP Top 10 framework, revealing both enhanced task performance and critical safety trade-offs. This work highlights the potential of LLMs to support MITM mitigation through adversarial awareness and informed response generation.

Also, [198] evaluated whether LLM-based agents, ranging from single-agent systems like GPT-4 and Claude to modular frameworks like PentestGPT, AutoAttacker, and Pen-Heal, can reliably execute multi-phase penetration testing, including real-time attacks such as MITM. The core problem identified is that current LLMs consistently fail to handle dynamic, timing-sensitive operations due to limitations in context retention, tool accuracy, and real-time responsiveness. To address this, the authors proposed five targeted architectural augmentations: Global Context Memory (GCM), Inter-Agent Messaging (IAM), Context-Conditioned Invocation (CCI), Adaptive Planning (AP), and Real-Time Monitoring (RTM). These interventions significantly improved task completion rates and mitigated failure modes, especially in modular agents, offering a promising pathway toward more autonomous and resilient LLM-driven cybersecurity systems.

Recent mitigation-oriented studies further expanded the threat model to cyber-physical and agentic settings. For instance, LLM-enabled robots and smart devices can be exposed to MITM-style command/telemetry manipulation, motivating defenses that combine authenticated channels, anomaly aware control policies, and “safety envelopes” on actions. As an example, MITM risks in LLM-enabled vacuum robots highlighted how prompt-to-actuation chains required integrity checks and stateful authentication, not only content filtering [199]. In parallel, closed-loop mitigation frameworks for next-generation networks proposed agentic pipelines that couple detection with constrained response actions (e.g., policy updates, routing adjustments, isolate/contain), which is particularly relevant for Open RAN/6G environments [196].

Overall, the common mitigation direction across MITM layers is *defense-in-depth*: (i) prevent MITM-enabling weaknesses in code/config (static auditing + taxonomy updates), (ii) harden LLM pipelines (message integrity, tool provenance, UI injection resistance), and (iii) enforce safe, bounded actions in closed-loop mitigation (rate limiting, rollback, human-in-the-loop for high-impact actions).

4.8.3 Comparative evidence

Table 13 summarizes recurring architectural patterns across MITM-related works, emphasizing how LLMs are integrated (classifier, auditor, agentic controller), their main strengths (semantic reasoning, policy alignment, closed-loop actions), and typical limitations (latency/cost, tool trust, adversarial promptability).

4.9 Summary

Building on Section IV, we consolidate the surveyed works into a unified evidence table XIV that supports cross-attack comparison even when experimental setups differ. In this table, we (i) explicitly tag each work by architectural pattern (P1–P5) and deployment regime (inline vs. offline), and (ii) separate the *LLM role* (reasoner, encoder, agent/tool-user, rule generator) from the security task (detection vs. mitigation). This makes the table actionable, i.e., readers can quickly identify which designs are feasible under strict latency constraints (inline IDS), and which are better suited for analyst path triage and response automation (Table 14).

5 Feasibility and deployment readiness

Section IV shows that reported gains depend as much on the *pipeline design* as on the LLM itself. Here, we evaluate *deployment readiness* through the lens of operational constraints (latency/throughput, SOC placement and integration, cost/privacy/governance, and domain shift) and LLM-specific risks (prompt injection, poisoned retrieval, hallucinations, and tool/agent misuse), together with practical assurance mechanisms.

5.1 System-level constraints and qualified claims

In this survey, we use *real-time* only when the end-to-end pipeline (feature extraction → model inference → decision) sustains the target event rate under bounded latency; otherwise we refer to *near-real-time* (seconds-to-minutes) or *analyst path* operation (interactive triage). Similarly, claims of *scalability* and *reduced false positives* are conditioned on (i) workload

Table 14 Unified cross-attack synthesis of representative LLM-based solutions

Attack	Ref.	Pattern	LLM Role	Input/Dataset	Task	Deploy.	Key takeaway
DoS/DDoS	[114]	P1	Generative reasoner (packet-to-text)	Network packets → text	Detect	Inline/nearline	Shows feasibility of textified packet reasoning with strong detection metrics under real-time constraints.
DoS/DDoS	[116]	P2	Security-specialized encoder	IoT/IoT traffic (PPFLE)	Detect	Inline/edge	Lightweight, privacy-preserving fixed-length encoding enables high-accuracy multi-attack detection on constrained devices.
DoS/DDoS	[118]	P4	Tool-using/role-conditioned agent	Simulated DDoS scenarios	Mitigate	Online	Agentic pipeline injects domain knowledge + roles to produce faster, adaptive mitigation with fewer false positives.
DoS/DDoS	[115]	P1/P4	Reasoner with progressive role reasoning	CICDDoS2019 flows	Mitigate/Detect	Nearline	Improves interpretability by embedding global+local evidence into structured reasoning for robust classification.
DoS/DDoS	[107]	P5	Multimodal contrastive pretraining	Encrypted traffic (packet+NL)	Detect	Nearline	Demonstrates that multimodal alignment can increase anomaly detection and interpretability under payload opacity.
Phishing	[125]	P1	Prompted classifier + explanations	Email corpora (structured prompts)	Detect	Nearline	Structured prompting enables high-accuracy phishing email detection with user-facing rationales.
Phishing	[126]	P5	Multimodal brand + domain reasoning	Webpage content + domain signals	Detect	Nearline	Two-stage MLLM pipeline improves robustness by combining brand inference and domain verification.

Table 14 continued

Attack	Ref.	Pattern	LLM Role	Input/Dataset	Task	Deploy.	Key takeaway
Phishing	[128]	P2/P5	Robustness evaluation (adversarial)	Phishing webpages (adversarial)	Detect	Offline	Highlights when LLM-based detectors fail under adversarial perturbations and what features improve resilience.
Phishing	[129]	P4	Multi-agent LLM detector	Email detection pipeline (agents)	Detect	Nearline	Uses agent decomposition to improve recall and explanation quality; best suited for analyst path workflows.
Phishing	[130]	P2	Quantized LLM vs classical models	Phishing detection benchmarks	Detect	Inline/edge	Shows feasibility frontier of compact/quantized LLMs under latency budgets compared to classical baselines.
SQLi	[147]	P4	LLM-assisted black-box testing	Web targets/payload generation	Detect	Offline/nearline	Evaluates LLMs for automated SQLi probing; improves payload search and coverage in black-box settings.
SQLi	[145]	P1/P2	LLM-based detector (queries/text)	SQLi traces/query text	Detect	Nearline	Demonstrates LLM-driven detection pipeline for SQLi patterns with improved generalization over handcrafted rules.
SQLi	[149]	P4	Adversarial generation (RL + LMs)	Adversarial SQLi payloads	Mitigate/Assess	Offline	Shows that stronger attack generation stress-tests defenses, motivating robust detection and safer query handling.
SQLi	[156]	P3/P4	Security analysis of Text-to-SQL	Text-to-SQL models (backdoor)	Assess/ Mitigate	Offline	Migrates SQLi threats into text-to-SQL via backdoors; motivates data sanitization and output constraints.
SQLi	[159]	P1/P4	Systematic gap analysis + mitigations	LLM-generated queries	Assess/ Mitigate	Offline	Identifies detection gaps in LLM-generated SQL; recommends stronger validation, parameterization, and guardrails.

Table 14 continued

Attack	Ref.	Pattern	LLM Role	Input/Dataset	Task	Deploy.	Key takeaway
Malware	[164]	P4	Hybrid fast path + LLM stage	npm packages + CodeQL prefilter	Detect	Nearline	Two-stage design reduces cost while improving precision/F1 over static-only baselines.
Malware	[165]	P2	Long-context encoder (syscalls)	Syscalls (large-scale)	Detect	Inline/nearline	Long-context transformers capture extended behavior dependencies for complex malware families.
Malware	[170]	P2/P4	Few-shot multi-label Android detection	Android apps (few-shot)	Detect	Nearline	Targets data-scarce regimes by combining LLM priors with few-shot learning for multi-label classification.
Malware	[200]	P5	LLM + GNN (ransomware)	Ransomware behavior graphs	Detect/ Mitigate	Nearline	Combines LLM semantics with structural reasoning to detect deep ransomware behaviors beyond signatures.
XSS	[178]	P4	LLM-assisted ML detection	XSS datasets/feature pipelines	Detect	Nearline	Uses LLM guidance to strengthen ML-based detectors via better feature reasoning and error analysis.
XSS	[183]	P1/P5	Semantic understanding for stored XSS	Stored XSS payload + context	Mitigate	Online	Defends against stored XSS by combining semantic interpretation with context-aware filtering.
XSS	[184]	P4	GenAI for WAF rule evolution	WAF logs/attack traces	Detect/ Mitigate	Nearline	Automates WAF-side detection updates for evolving XSS payloads via generative rule adaptation.
Eavesdrop.	[187]	P4	Policy-enforcing agentic privacy monitor	Multi-agent communications	Detect/ Prevent	Online	Reference monitors + quarantined local LLM prevent sensitive leakage in agent interactions.
Eavesdrop.	[186]	P5	LLM-augmented acoustic inference attack	mmWave radar + audio signals	Threat model	Offline	Shows emerging eavesdropping surfaces where LLMs amplify inference from heterogeneous sensory traces.

Table 14 continued

Attack	Ref.	Pattern	LLM Role	Input/Dataset	Task	Deploy.	Key takeaway
Eavesdrop.	[191]	P1/P4	Privacy defense vs ASR (incl. LLM-powered)	Speech privacy benchmarks	Mitigate	Online	Protects user privacy against commercial and LLM-powered ASR, motivating stronger privacy-by-design controls.
MITM	[192]	P2/P4	LLM-based code auditor + taxonomy	Crypto misuse in code	Detect	Offline	Automatically expands misuse taxonomy linked to MITM vectors (cert/hostname verification failures).
MITM	[194]	P4	Message-layer tampering analysis	LLM-MAS communication	Threat model	Online	Shows stealthy multi-round tampering, motivating message integrity checks and cross-agent consistency tests.
MITM	[195]	P4	Toolchain hardening (provenance)	Tool metadata/tool invocation	Mitigate	Online	Demonstrates that tool metadata can induce unsafe actions, motivating signed schemas and least-privilege tools.

regime (inline vs. offline), (ii) deployment budget (compute, batching, caching), and (iii) evaluation realism (class imbalance, drift, encryption, adversarial adaptation).

5.1.1 Latency and throughput (Tokens are the new budget)

LLM-based defenses add tokenization and generation (or long-context attention) costs that often dominate compute in prompt-based pipelines (Pattern P1) and tool-using agents (Pattern P4). Even when framed as classification, “telemetry textification” and long prompts can inflate latency under high-volume traffic. Therefore, deployment-relevant “real-time” claims should report at minimum: (a) representation size (tokens/event), (b) batching/caching strategy, (c) achieved throughput (tokens/s or events/s), and (d) tail latency (p95/p99) under realistic bursts.

5.1.2 Cost envelope and hardware dependence:

Compact encoders (Pattern P2) can often run on commodity CPUs/edge accelerators, whereas prompt-heavy or agentic pipelines typically require GPU-class inference to meet throughput targets. This shifts feasibility from “can it detect?” to “can it detect within budget?”. To make claims comparable, we recommend reporting a normalized metric such as “cost per million events” alongside accuracy/F1 and latency statistics.

5.1.3 Edge/IoT vs. centralized cloud/SOC:

To avoid conflating evaluation settings, we explicitly distinguish two deployment regimes: (i) *Edge/IoT/IIoT deployments* operate under strict resource constraints (limited CPU/GPU/accelerator availability, memory, and energy budgets), favoring compact encoders and lightweight detection models that can run locally and transmit only summarized evidence. (ii) In contrast, *centralized cloud/SOC deployments* can leverage stronger compute and broader telemetry aggregation, making them more compatible with retrieval-augmented reasoning and tool-integrated workflows (e.g., SIEM/SOAR/EDR enrichment), albeit with stricter throughput and tail-latency requirements at scale. Accordingly, we annotate IDS approaches by their practical placement, i.e., edge-suitable methods (lightweight/compact models, fast path filters) versus centralized methods (LLM reasoning, RAG, and agentic tool use) and motivate cascaded pipelines where edge components triage and escalate only suspicious cases to a centralized LLM.

5.1.4 Failure under surge traffic

Under traffic spikes, LLM stages can become the bottleneck (backpressure/queues), while lossy representations can drop timing/protocol cues needed for volumetric attacks (e.g., DoS/DDoS), and brittle prompting can change outputs with minor template edits. These constraints motivate staged designs, where a fast path filter (signatures/compact ML) handles the bulk stream, while an LLM is reserved for a small suspicious subset for semantic triage, explanation, and guided investigation.

5.1.5 What “false positives” mean in practice

In operational SOC settings, “false positives” often manifest as *analyst-visible noise* rather than only the statistical FP rate. LLMs may reduce burden by clustering alerts, summarizing evidence, and improving traceability, but can also introduce new error sources (hallucinated rationales, prompt injection, poisoned retrieval). Hence, we distinguish (i) detection FP rate (model-level), (ii) alert fatigue (workflow-level), and (iii) mitigation safety (action-level).

5.2 Operational Placement for Latency, Throughput, and Evidence Budgeting

5.2.1 Inline versus analyst path placement

Across attack families, *inline* deployment is rarely limited by accuracy alone, but by throughput and tail latency. Inline IDS/IPS workloads operate under tight budgets (often milliseconds per decision at scale), favoring compact discriminative encoders (Pattern P2) or two-stage cascades (fast path filter + slow path LLM reasoning). Recent work showed hybrid deployment-ready pipelines remain strong baselines. For instance, a Transformer-based IoT IDS couples efficient representation learning with classical detection, and Android malware ensembles match competitive accuracy with lower inference cost than large generative models [201, 202]. Accordingly, LLMs can be framed as complementary in a staged design, hybrid detectors handle high-volume telemetry (fast path), while LLMs are invoked selectively for semantic triage, grounded explanations, and SOC workflow support under tight token/latency budgets. In contrast, analyst path workflows (triage, investigation, reporting) tolerate higher latency and benefit from generative reasoning, RAG grounding (Pattern P3), and tool-using agents (Pattern P4).

5.2.2 A two-lane design (fast path + slow path)

A pragmatic architecture aligned with Dimension D1 (inline vs. offline deployment) separates:

- **Lane A (fast path):** signatures and/or lightweight models process the bulk stream, producing alerts plus compact evidence summaries.
- **Lane B (slow path):** an LLM (optionally agentic) is invoked only for a minority of suspicious cases to provide semantic interpretation, rationale, and next-step guidance.

This separation stabilizes latency under surge traffic, reduces token cost, and avoids forcing the LLM into an infeasible line-rate role.

Although large models (e.g., GPT-4 or LLaMA-70B-class) often report strong detection performance in prompt-based settings, they typically require GPU-class inference and incur higher latency and cost due to long-context prompting and generation. In contrast, lightweight encoders (e.g., DistilBERT) and security-specialized compact models (e.g., SecurityBERT) are more compatible with SOC-scale throughput requirements and resource-constrained environments (e.g., IoT/edge), at the cost of reduced general-purpose reasoning and limited tool-use capabilities. Accordingly, this survey frames model selection as a deployment trade-off between accuracy gains and operational budgets (tokens/event, throughput, and tail latency) and motivates cascaded “fast path + slow path” designs in which lightweight models handle the bulk stream, while LLMs are reserved for a minority of suspicious cases requiring semantic triage and explanation.

5.2.3 Evidence schemes and telemetry compression

Patterns P1 and P5 imply that raw artifacts (packets/flows/HTML/logs) must be compressed into bounded representations. A deployment-ready pipeline defines an *evidence schema* (key-value summaries, structured fields, typed graphs) rather than free-form dumping of logs into prompts. The schema should preserve the signal that matters for the family. For example, for DoS/DDoS, rate/timing statistics matter most, but for phishing, semantic and layout cues are the most important.

5.2.4 Token budgets and controlled context growth

Feasibility depends on end-to-end token budgets, linked to preprocessing (textification/summarization), inference (prompt + retrieved context), and post-processing (verdict/action drafting). To avoid runaway context growth, we recommend bounded evidence windows, de-duplication of repeated fields, hierarchical summarization over time slices, and selective retrieval scoped to the current decision.

5.3 Integration with SIEM/SOAR/EDR and SOC Workflows

5.3.1 How LLMs can be integrated

LLM-enabled defenses become actionable when integrated with operational systems, including SIEM (aggregation/search), EDR (endpoint telemetry), and SOAR (playbooks/case management). Across surveyed patterns, we observe three high-value integration points:

- 1) **Query and enrichment copilot:** i.e., generate/repair SIEM queries (e.g., KQL/SPL), map indicators to ATTACK tactics, and propose pivots from limited evidence.
- 2) **Alert summarization and case narrative:** by converting multi-alert evidence into structured incident summaries (“what happened/why it matters/what to do next”).
- 3) **Guarded playbook drafting:** e.g., draft SOAR steps (containment, notification, evidence preservation) subject to strict policy checks and approval.

5.3.2 LLM functions for integration

Tool-using/agentic pipelines for triage and response (Pattern P4) provide strong productivity gains but introduce the highest safety risk. Hence, a deployment-ready design uses the following staged permissions:

- **Tier 0:** summarize, classify, and recommend actions (no tool calls).
- **Tier 1:** read-only tools (search, query, fetch logs, enrich IOCs).
- **Tier 2:** restricted actions (isolate host, block domain, revoke tokens) only via allow-listed actions and human approval.

5.4 Integration use case: phishing triage → SIEM Pivots → SOAR Draft

To make the deployment discussion concrete, consider a phishing email alert ingested by the SIEM (suspicious sender domain + embedded URL). An LLM improves analyst throughput only if embedded in a constrained workflow with explicit evidence boundaries and tool controls. Specifically, the following steps can be followed:

Table 15 Integration use case checkpoints: Where risks enter and where controls attach

Stage	Primary risk	Control
Alert summary	injection via email body	schema separation + sanitization
SIEM pivots	unsafe queries/leakage	read-only tools + audit logs
RAG enrich	poisoned retrieval	allow-listed corpora + provenance
SOAR draft	unsafe actions	approval gates + least privilege

- *Step 1: Tier 0 summarization with instruction–data separation*: Raw artifacts (headers, URL, body snippets, attachment hashes) are converted into a typed evidence schema (Pattern P1). The LLM outputs: (i) a short synopsis, (ii) candidate IOCs, and (iii) explicit “missing evidence” requests (e.g., URL detonation verdict). Crucially, the email body remains *untrusted data* and cannot override system instructions.
- *Step 2: Tier 1 SIEM pivots as read-only tool calls* The LLM proposes SIEM pivots (e.g., “who clicked the URL?”, “other recipients?”, “similar subjects in last 24 h?”, “DNS lookups?”). Queries execute via read-only connectors; then, results return as evidence IDs (event IDs + time windows) rather than free text.
- *Step 3: Tier 1 grounded reasoning with controlled retrieval (Pattern P3)*: If retrieval is used, only allow-listed corpora (internal IR playbooks, vetted CTI) are queried. Each claim must cite evidence IDs. Otherwise, the model abstains. This counters hallucinated IOCs and limits poisoned retrieval/instruction injection.
- *Step 4: Tier 2 SOAR plan drafting with approval gates*: The model drafts a response plan but cannot execute actions. High-impact steps (block domain, isolate host, revoke tokens) require policy checks, human approval, and an auditable record of tool calls and evidence.

This workflow surfaces key misuse vectors, including prompt injection in the email body, poisoned retrieval via compromised playbooks/CTI, hallucinated causal claims, and tool/agent coercion. Schema separation, provenance labeling, evidence-linked claims, staged tool access, and approval gates can provide containment strategies [203, 204]. The key checkpoints for the use case are summarized in Table 15.

5.5 Engineering realism: from prototype to deployable SOC component

To improve applicability, we translate surveyed patterns into engineering requirements. Most LLM-based defenses are not deployable as inline IDS replacements. They become realistic when framed as constrained SOC components with explicit Service Level Objectives (SLOs), bounded context, and safe fallback behavior.

5.5.1 SLOs definition

A deployment-ready pipeline should report service-level objectives and workloads, including (i) event rate (events/s), (ii) max evidence size (tokens/event), (iii) tail latency targets (p95/p99), and (iv) queue/backpressure policy.

5.5.2 Capacity budgeting and graceful degradation

Systems should enforce hard caps on evidence windows, de-duplication, and retrieval limits. When budgets are exceeded (bursts, long investigations), the pipeline should degrade safely (more aggressive summarization, throttled retrieval/tool calls, abstention, analyst routing).

5.5.3 Real system requirements

Beyond the model, deployable systems require evidence schemes, tool contracts (allow-listed connectors with strict I/O schemas and tamper-evident audit logs), fallback policies (abstention + analyst routing), and operational monitoring (abstention rate, tool-call rate, disagreement with fast path detectors, drift indicators).

5.5.4 Evaluation protocol for practical deployments:

We recommend three types of metrics: (i) offline metrics (accuracy/F1), (ii) shadow deployment (measure alert volume, time-to-triage, injection/hallucination incidence), and (iii) adversarial tests (prompt injection, retrieval poisoning, tool-misuse attempts) with enforced safe failure.

5.6 Cost, privacy, and governance

5.6.1 Cost at SOC scale

Even when per-request inference is affordable, continuous monitoring and repeated investigations multiply cost. Two-stage cascades, evidence schemas, and caching (e.g., repeated phishing templates) reduce unnecessary LLM calls.

5.6.2 Privacy and data residency

Security telemetry may include credentials, personal identifiers, and proprietary logs. Deployment choices (on-prem LLM vs. managed APIs) must be driven by data sensitivity and regulation. We recommend redaction of secrets/PII before any LLM call, tenant-isolated storage for prompts/retrieved context, and explicit retention policies for incident response transcripts.

5.6.3 Governance artifacts

For SOC adoption, each pipeline should include a threat model (prompt/retrieval/tool attack surfaces), a model card (limitations/intended use), and audit logs linking outputs to evidence and tool calls.

5.7 Domain shift and evaluation realism

5.7.1 Concept drift is the norm

Real deployments face shifts in traffic mix, user behavior, software versions, and attacker tactics, so offline benchmarks can overestimate robustness. We recommend “shadow mode” deployment alongside existing IDS/EDR before enabling partial automation.

5.7.2 Calibration and abstention:

LLM outputs should be treated as *advice*, not truth. A practical mechanism is calibrated abstention: when evidence is insufficient or uncertainty is high, return “needs analyst review” and request specific missing fields.

5.8 LLM-specific risks and assurance mechanisms

5.8.1 Risk surface

LLM-based cyber defense introduces failure modes beyond classical ML, including prompt injection/instruction hijacking, poisoned retrieval in RAG, hallucinations, and tool/agent misuse.

The risk is not only model error, but also how errors propagate through decision-making. Fluent narratives can induce automation bias. Hence, deployment-ready designs should make uncertainty explicit and force evidence-linked claims such that analysts can verify conclusions quickly.

5.8.2 Runtime monitoring and safe failure

Pipelines should monitor abstention rate, tool-call rate, disagreement with fast path detectors, and drift indicators. When thresholds are exceeded (surge traffic, anomalous tool use), the system should degrade safely, through shortening context, limiting tools/retrieval, and routing to analysts.

5.8.3 Assurance toolbox and risk-to-control mapping:

We synthesize defenses into five categories (mapped to Patterns P1–P5):

- 1) *Structured interfaces*: separate *instructions* from *untrusted data* via typed schemas and strict parsers; reject non-conforming outputs.
- 2) *Sanitization and provenance*: strip/escape instruction-like tokens, attach source/time/trust labels, and constrain how retrieved text influences decisions.
- 3) *Grounded generation*: require evidence IDs for claims; force abstention when grounding is missing.
- 4) *Constrained tools*: allow-lists, least privilege, rate limits, and human approval for high-impact actions; log every call.
- 5) *Continuous red teaming*: adversarial suites for injection, retrieval poisoning, and agent manipulation.

Table 16 maps risks to entry points and controls.

Table 16 Deployment risks and assurance mechanisms for LLM-based cyber defense

Risk	Where it enters	Recommended controls
Prompt injection	emails/webpages/logs (P1)	sanitize untrusted text; schema separation; abstention on conflicts
Poisoned retrieval	CTI/playbooks/tickets (P3)	provenance scoring; retrieval filtering; instruction-blocking wrappers
Hallucinations	generation step	evidence-linked claims; grounded summaries; post hoc verifiers
Agent/tool misuse	tool calls (P4)	allow-lists; least privilege; approval gates; full audit logs

5.9 Takeaways

Across attack families, the most reliable deployments treat LLMs as *bounded components* inside audited pipelines, where compact models handle volume, LLMs handle semantics and operator interaction, and assurance mechanisms constrain how untrusted inputs, retrieved text, and tools affect outcomes. This shifts evaluation from “Does the model achieve high performance?” to “Does the *system* remain correct, safe, and cost-feasible under drift and adversarial pressure?”.

6 Open issues and future directions

Despite significant progress, several technical, operational, and ethical challenges must be addressed before LLMs can be fully and safely integrated into network security systems. Recent position paper [205] on trustworthy and responsible LLM deployment emphasized that transparency, explainability, robustness, and accountability are essential prerequisites for deploying LLMs in safety-critical domains such as cybersecurity. Building on these insights, future LLM-driven intrusion detection and attack mitigation systems should integrate built-in interpretability mechanisms, hallucination control strategies, and bias-aware training pipelines to ensure operational reliability and analyst trust. In particular, security-oriented LLM frameworks must incorporate verification layers, confidence estimation, and adversarial robustness to mitigate risks associated with black-box decision-making and model manipulation. Accordingly, future research should focus on addressing these issues.

6.1 Benchmark datasets and evaluation gaps for LLM-based cyber defense

Despite the availability of multiple benchmark datasets for network intrusion detection and security analytics [206, 207], important limitations remain when these resources are used to evaluate LLM-based cyber defense pipelines. Most existing datasets were originally designed for classical machine learning or deep learning classifiers operating on structured network features or packet-level statistics. As a result, they often lack the heterogeneous, multi-modal, and context-rich evidence that LLM-driven systems are designed to process, such as incident narratives, alert correlations, threat intelligence reports, code artifacts, and cross-source telemetry.

A major challenge is the limited realism of many benchmark environments. Synthetic traffic generation, constrained background noise, and simplified attack scenarios may lead to overly optimistic performance estimates that do not transfer to operational SOC settings. In particular, real-world deployments involve mixed benign and malicious traffic, evolving user behavior, encrypted communication, and adversarial adaptation, which are rarely captured simultaneously in public datasets. This gap is especially critical for evaluating end-to-end detection and mitigation pipelines under realistic latency and throughput constraints.

From an evaluation perspective, current studies predominantly rely on classification-oriented metrics such as accuracy, precision, recall, and F1-score. While these metrics are informative for isolated detection tasks, they provide limited insight into operational effectiveness. For LLM-driven security pipelines, additional dimensions should be considered, including detection latency, alert volume reduction, analyst workload impact, false-positive cost, response execution time, and overall incident resolution effectiveness. These metrics better reflect SOC feasibility and real-world deployment constraints.

Furthermore, explainability and trust-related indicators remain underexplored in quantitative evaluations. Future benchmarks should incorporate measures of explanation quality, evidence traceability, robustness to prompt manipulation, and resilience against adversarial inputs. Such metrics are essential for assessing the risk and assurance aspects of LLM-based cyber defense, which are critical for adoption in regulated and safety-critical environments.

Advancing LLM-based cybersecurity research requires not only larger datasets, but also more representative, multi-modal benchmarks and standardized evaluation protocols that capture detection performance, mitigation effectiveness, operational scalability, and trust-worthiness. Establishing such benchmarking frameworks represents a key research direction for the community.

6.2 Dataset limitations and evaluation metrics

Many available datasets are outdated, unbalanced, or lack realism. There is a need for updated, multimodal, and adversarially enriched datasets that better reflect current threat landscapes. Standardized evaluation metrics and benchmarking protocols should also be established. Besides that, language coverage is also a practical limitation. Recent studies on cyberbullying detection highlight the scarcity of resources and the need for tailored models in low-resource languages [208].

Another limitation concerns dataset bias and domain shift. Models trained and evaluated on a single benchmark often exhibit degraded performance when deployed across different networks, organizations, or traffic profiles. Future research should therefore prioritize cross-dataset evaluation protocols and domain-adaptive benchmarking to assess the generalization capabilities of LLM-based systems beyond narrow experimental settings.

6.3 Scalability and Real-Time Deployment

LLMs require substantial computational resources, which limit their use in real-time or edge-based security systems. Efficient versions of LLMs should be developed using pruning, quantization, distillation, or knowledge transfer to enable deployment in latency-sensitive and low-resource environments. Developing lightweight and energy-efficient architectures is essential for sustainable deployment. Models should also be evaluated based on environmental impact.

6.4 Robustness against adversarial attacks

LLM-based cybersecurity pipelines introduce a broader adversarial surface than conventional ML/DL IDS. Beyond classical evasion and data poisoning (well studied for CNN/RNN/autoencoder detectors operating on numeric flow features), LLM-enabled defenses must withstand *instruction-level* and *tool-level* threats. In particular, attackers can attempt (i) **prompt injection** and instruction hijacking through untrusted artifacts (emails, webpages, logs, tickets), (ii) **jailbreaks** that circumvent safety policies and induce unsafe recommendations, (iii) **retrieval poisoning** in RAG settings where compromised CTI/playbooks bias the model's conclusions, and (iv) **agent/tool misuse** where the model is coerced into unsafe or costly actions (e.g., generating overly broad SIEM queries, leaking sensitive context, or triggering disruptive playbooks).

A core robustness requirement is to separate *trusted instructions* from *untrusted evidence* via typed schemas (structured fields for headers/IOCs/telemetry summaries) and strict parsing. Untrusted text should be sanitized (escaping instruction-like patterns) and labeled with provenance so it cannot override system prompts. This control is largely orthogonal to non-LLM DL IDS, which typically do not execute instruction-following behavior at inference time.

For RAG-based defenses, retrieval must be constrained to allow-listed corpora with provenance scoring and filtering. To reduce hallucinations and poisoned-retrieval effects, model outputs should require evidence-linked claims (e.g., cite event IDs / retrieved document IDs) and *abstain* when grounding is missing. This shifts robustness evaluation from “accuracy under perturbations” to “faithfulness under adversarial context.”

We recommend expanding robustness evaluation beyond standard evasion benchmarks to include (i) prompt-injection suites on realistic artifacts (phishing bodies, web content, noisy logs), (ii) retrieval-poisoning tests (malicious/compromised CTI or playbooks), (iii) jailbreak and policy-violation probes, and (iv) agentic misuse tests (coercing unsafe tool calls). A deployment-ready system should demonstrate *safe failure* (bounded outputs, abstention, and analyst routing) under these stressors rather than only maintaining accuracy.

Importantly, recent non-LLM (or hybrid) defensive frameworks already illustrate the value of *explicit adversarial evaluation* as a first-class requirement. For instance, *StealthPhisher* reports high phishing detection performance and evaluates robustness under FGSM-style adversarial perturbations while using GenAI for explainable, user-facing reporting [209]. Similarly, *PermGuard* proposes a scalable Android malware detection pipeline with explicit resilience testing against adversarial attempts and strong accuracy on large balanced datasets, supporting deployability under evolving threat conditions [210].

In this context, promising directions include adversarial training or red-teaming-driven fine-tuning for injection resilience, input filtering and provenance-aware prompting, verifier models for grounded summaries, and reinforcement learning or policy constraints for tool use.

6.5 Explainability and interpretability

LLMs (and transformer-based security models more broadly) can deliver highly accurate predictions without providing a transparent rationale for *why* a flow/log/session is flagged. This black-box behavior remains a major barrier to adoption in security-critical and regulated environments, where analysts must validate alerts, justify mitigation actions, and perform root-cause analysis. A key research direction is therefore to integrate Explainable AI (XAI) into LLM/transformer-driven detection pipelines so that decisions are accompanied by clear, human-interpretable evidence (e.g., feature attributions, local explanations, and saliency over structured telemetry). Recent IDS work such as DistilXIDS illustrated this direction by embedding post hoc interpretability (SHAP and LIME) into a distilled transformer (DistilBERT) for real-time DDoS detection, explicitly aiming to improve analyst trust and operational transparency while maintaining strong predictive performance and deployment readiness [80]. A similar transparency-driven trend is also observed in text-centric security tasks, where for instance, transformer-based SMS spam detection has been enhanced with XAI methods that expose evidence for model decisions (e.g., via positive/negative contribution scores), improving interpretability while preserving very high detection accuracy [211].

7 Conclusion

This survey provided a detailed analysis of the use of LLMs in network security, focusing on their capabilities to detect and mitigate a wide range of cyberattacks, including DoS/DDoS, phishing, malware, SQL injection, cross-site scripting, and eavesdropping. Beyond summarizing prior work, our key contribution is a synthesis-driven view of *how* LLMs are being operationalized in cybersecurity pipelines (from evidence representation to decision and response), and *where* they are realistically deployable (e.g., inline vs. analyst path) under SOC constraints. LLMs demonstrate strong potential in learning from large-scale, unstructured data and capturing complex attack patterns that traditional methods often fail to identify. They outperform classical rule-based systems by enabling contextual analysis, real-time threat detection, automatic rule generation, and adaptive response strategies. Several research works reviewed in this survey show the high accuracy and robustness of LLM-based models in diverse attack scenarios, including SecurityBERT for resource-constrained IoT environments, LLMSQLi for black-box SQL injection detection, and ChatSpamDetector for phishing email classification. Importantly, we do not treat these results as purely model-centric. Instead, we highlight that reported gains depend on pipeline design choices (e.g., prompt-based vs. encoder-based approaches, grounding via retrieval, and tool-assisted workflows) and on evaluation realism (drift, class imbalance, encrypted traffic, and adaptive adversaries). Despite these promising outcomes, LLMs introduce challenges related to interpretability, high computational demands, vulnerability to adversarial inputs, and potential misuse. Some models are also susceptible to prompt injection and DoS attempts that exploit their own safety mechanisms. Accordingly, this survey contributes a structured critique of LLM-specific risks (prompt injection, poisoned retrieval, hallucinations, and tool/agent misuse) and consolidates practical assurance mechanisms (schema separation, provenance/grounding, constrained tool access, and safe fallback/abstention) needed for trustworthy adoption. Addressing these limitations requires the development of lightweight architectures, enhanced explainability, adversarial robustness, and secure integration strategies. Emerging solutions like fine-tuning, reinforcement learning from human feedback (RLHF), and structured prompting are helping tailor models to security-specific needs. LLMs are not intended to replace conventional cybersecurity tools but to augment them by providing scalable, automated, and intelligent support. Their real value lies in enabling faster response, deeper inspection of network behaviors, and continuous learning from evolving threats. To support researchers and practitioners, we also distill actionable guidance on (i) evidence budgeting and token-cost realism, (ii) integration points with SIEM/SOAR/EDR, and (iii) evaluation regimes that move from offline metrics to shadow deployment and adversarial testing. To ensure safe and efficient deployment, future research should prioritize standard evaluation benchmarks, model transparency, resource optimization, and secure multimodal frameworks. When integrated thoughtfully, LLMs can significantly enhance the resilience and efficiency of modern cybersecurity systems.

Author Contributions M.B. = Mariem Ben Ali; M.A. = Mirna Awad; W.J. = Wael Jaafar; L.C.F. = Lamia Chaari Ep Fourati. L.C.F and W.J. conceived and led the study. W.J. and L.C.F. provided supervision and critical guidance. M.B. and M.A. designed the survey protocol and inclusion/exclusion criteria for the initial submitted version. M.B. and M.A. performed the literature search, screening, and data curation. LCF synthesized findings and developed the taxonomy with input from W.J. M.B prepared the figures and tables. M.B. drafted the initial version of the manuscript; M.A., W.J., and L.C.F. reviewed and edited the manuscript for important intellectual content. L.C.F and W.J. revised the manuscript according to the reviewers comments.

Data Availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Bouraffa T, Hui K (2025) Regulating information and network security: review and challenges. *ACM Comput Surv* 57(5):1–38
2. Goffer MA, Uddin MS, Hasan SN, Barikdar CR, Hassan J, Das N, Chakraborty P, Hasan R et al (2025) Ai-enhanced cyber threat detection and response advancing national security in critical infrastructure. *J Posthumanism* 5(3):1667–1689
3. Zou Y (2025) Design and implementation of computer network security monitoring system. *J Comput Signal Syst Res* 2(4):63–68
4. Guanqiao Q, Chen Q, Wei W, Lin Z, Chen X (2024) and Kaibin Huang. A contemporary survey, Mobile edge intelligence for large language models
5. Sakib Md N, Islam Md A, Pathak R, Arifin Md M (2024) Risks, causes, and mitigations of widespread deployments of large language models (LLMs): a survey
6. large language models and next-generation cybersecurity (2025) Aitizaz Ali and Mohamed C Ghanem. Beyond detection SHIFRA 81–97
7. Houssel PRB, Layeghy S, Singh P, Portmann M (2026) ex-nids: a framework for explainable network intrusion detection leveraging large language models. *Comput Electr Eng* 129:110826
8. Feng Y, Sakurai K (2025) Network intrusion detection: evolution from conventional approaches to IIM collaboration and emerging risks. *arXiv preprint [arXiv:2510.23313](https://arxiv.org/abs/2510.23313)*
9. Tianxing F, Jia H, Min G, Khowaja SA, Singh K, Dev K (2025) Federated retrieval-augmented generation-based LLM for enhanced cyber threat detection in the internet-of-energy. *IEEE Network*
10. Rustam F, Obaidat I, Monda D D, Jurcut A D (2025) Few-shot retrieval-augmented LLMs for anomaly detection in network traffic. In: *International conference on cryptology and network security*, Springer, pp 631–642
11. Dong Y, Aung YL, Chattopadhyay S, Zhou J (2025) Chatiot: Large language model-based security assistant for internet of things with retrieval-augmented generation. *arXiv preprint [arXiv:2502.09896](https://arxiv.org/abs/2502.09896)*
12. Yang S, Zheng X, Zhang X, Xu J, Li J, Xie D, Long W, Ngai E C (2025) Large language models for network intrusion detection systems: foundations, implementations, and future directions. *arXiv preprint [arXiv:2507.04752](https://arxiv.org/abs/2507.04752)*
13. Sharma AN, Akbar K A, Thuraishingham B, Khan L (2025) Enhancing security insights with knowgenrag: Combining knowledge graphs, LLMs, and multimodal interpretability. In: *D Proceedings of the 2025 ACM international workshop on security and privacy analytics 2*, pp 2–12
14. Zhang J (2025) Understanding and managing integrated advanced persistent threats: evaluation and analysis of large language models for phishing detection in small and midsize enterprises. PhD thesis, University of Bridgeport
15. De Queiroz, HJDS (2025) Phishing and social engineering attack prevention with LLMs. In: *Revolutionizing cybersecurity with deep learning and large language models*, IGI Global Scientific Publishing, pp 133–164
16. Liu H, Xue J, Zhao S, Liu Y, Lu Z (2025) The dual role of large language models in network security: survey and research trends. In: *Proceedings of the 2025 ACM workshop on wireless security and machine learning*, pp 20–25
17. Guo W, Wen H, Kong L, Xue J, Hu J, Han W, Wang Y (2025) A survey on malware analysis with large language models. In: *International conference on knowledge science, engineering and management*, Springer, pp 41–52
18. Wu D, Peng Z, Chen M, Liu Y (2025) Transforming network intrusion detection using large language models. In: *2025 IEEE 22nd consumer communications and networking conference (CCNC)*, IEEE, pp 1–9
19. Belcastro L, Carlucci C, Cosentino C, Liò P, Marozzo F (2025) Enhancing network security using knowledge graphs and large language models for explainable threat detection. *Futur Gener Comput Syst* 108160
20. Long S, Tan J, Mao B, Tang F, Li Y, Zhao M, Kato N (2025) A survey on intelligent network operations and performance optimization based on large language models. *IEEE Communications Surveys and Tutorials*

21. Li Y, Wen H, Wang W, Li X, Yuan Y, Liu G, Liu J, Xu W, Wang X, Sun Y, et al (2024) Personal llm agents: Insights and survey about the capability, efficiency and security. arXiv preprint [arXiv:2401.05459](https://arxiv.org/abs/2401.05459)
22. Chen J, Zhou H, Mei Y, Adam G, Bastian ND, Lan T (2023) Real-time network intrusion detection via decision transformers. arXiv preprint [arXiv:2312.07696](https://arxiv.org/abs/2312.07696)
23. Liu W (2025) Dynamic network intrusion detection model based on transformer and adversarial autoencoder. *International Journal of Intelligent Networks*
24. Atakishiyev S, Babiker HKB, Dai J, Farruque N, Hayashi T, Hriti N S, Md Abed Rahman, Iain Smith, Mi-Young Kim, Osmar R Zaiane, et al (2025) Explainability of large language models Opportunities and challenges toward generating trustworthy explanations arXiv preprint [arXiv:2510.17256](https://arxiv.org/abs/2510.17256),
25. Bian S, Yan M, Venkataraman S (2025) Scaling inference-efficient language models. arXiv preprint [arXiv:2501.18107](https://arxiv.org/abs/2501.18107)
26. Fernandez J Na C, Tiwari V, Bisk Y, Luccioni S, Strubell E (2025) Energy considerations of large language model inference and efficiency optimizations arXiv preprint [arXiv:2504.17674](https://arxiv.org/abs/2504.17674),
27. Chu J, Liu Y, Yang Z, Shen X, Backes M, Zhang Y (2025) Jailbreakradar: Comprehensive assessment of jailbreak attacks against LLMs. In: *Proceedings of the 63rd annual meeting of the association for computational linguistics vol 1, Long Papers*, pp 21538–21566
28. Li Z, Chen K, Liu L, Bai X, Yang M, Xiang Y, Zhang M (2025) Tf-attack: Transferable and fast adversarial attacks on large language models. *Knowl-Based Syst* 312:113117
29. Saied M, Adjogble F, Guirguis S, Hemmi M, Warschat J (2023) A framework for systematic scientific research management. In: *2023 Portland international conference on management of engineering and technology (PICMET)*, IEEE, pp 1–16
30. Chen Y, Cui M, Wang D, Cao Y, Yang P, Jiang B, Zhigang L, Liu B (2024) A survey of large language models for cyber threat detection. *Comput Secur* 145:104016
31. da Silva GDJC, Westphall CB (2024) A survey of large language models in cybersecurity. arXiv preprint [arXiv:2402.16968](https://arxiv.org/abs/2402.16968)
32. Hassanin M, Moustafa N (2024) A comprehensive overview of large language models (LLMs) for cyber defences: opportunities and directions. arXiv preprint [arXiv:2405.14487](https://arxiv.org/abs/2405.14487)
33. Yao Y, Duan J, Kaidi X, Cai Y, Sun Z, Zhang Y (2024) A survey on large language model (LLM) security and privacy: the good, the bad, and the ugly. *High-Confidence Computing* 100211
34. Kheddar H (2025) Transformers and large language models for efficient intrusion detection systems: a comprehensive survey. *Inf Fus* 124:103347
35. Habibzadeh A, Feyzi F, Atani R E (2025) Large language models for security operations centers: a comprehensive survey. arXiv preprint [arXiv:2509.10858](https://arxiv.org/abs/2509.10858)
36. Academia.edu. Introduction to network security, attacks, and services. https://www.academia.edu/9860698/Introduction_to_Network_Security_Attacks_and_Services, (2024)
37. Möller DPF (2026) *Cybersecurity for network and information security: principles. Techniques and Applications*, Springer Nature
38. Osazuwa CM (2024) Confidentiality, integrity, and availability in network systems: a review of related literature. *International Journal of Innovative Science and Research Technology*
39. Khanna A (2025) Introduction to cybersecurity. In: *Securing an enterprise: maximizing digital experiences through enhanced security measures*, Springer, pp 3–18
40. Möller DPF (2023) Nist cybersecurity framework and mitre cybersecurity criteria. In: *Guide to cybersecurity in digital transformation: trends, methods, technologies, applications and best practices*, Springer, pp 231–271
41. Parmar M, Miles A (2024) Cyber security frameworks (CSFs): an assessment between the nist csf v2.0 and eu standards. In *2024 Security for Space Systems (3S)*, IEEE, pp 1–7
42. Edwards J (2024) *A comprehensive guide to the NIST cybersecurity framework 2.0: strategies, implementation, and best practice*. 1 edn, Wiley
43. Megarry M, Sharma V, Canberk B, Duong TQ (2025) Detection of UE metric data poisoning in o-ran: automated risk analysis and resilience. *IEEE Communications Standards Magazine*
44. Si B, Sun H, Shao M (2025) Data-driven Bayesian network for risk analysis of telecom fraud. *Comput Inf* 44(4):800–827
45. Vaibhav Anil Vora (2025) Demystifying zero trust security: the no-trust network paradigm. *J Comput Sci Technol Stud* 7(3):141–148
46. Sinan M (2025) *Support security control management and implementation in DevSecOps*. PhD thesis, RMIT University
47. Ansari MS, Rajan UC (2026) *Cybersecurity risk management in it projects: a systematic review and classification of risks, best practices, and framework effectiveness (2020-2025)*. Best practices, and framework effectiveness (2020–2025) (January 07, 2026)

48. Kirubavathi G, Pulliyasseri A, Rajesh A, Ajayan A, Alfarhood S, Safran M, Alfarhood M, Shin J (2025) Enhancing IoT resilience at the edge: a resource-efficient framework for real-time anomaly detection in streaming data. *Comput Model Eng Sci* 143(3):3005
49. HowtoNetwork (2024) Network security design
50. Cloudflare. What is a ddos attack? <https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/>, n.d
51. OWASP (2024) Denial of service
52. Yergatti A RR, Shiggavi P, Adhoni MA, Boodihal SV (2025) Malware attack detection using network flows with machine learning. In: *International conference on smart trends for information technology and computer communications*, Springer, pp 37–48
53. Rao Y S, Sahu DK, Dihudi B, Kumar J, Dash AN (2025) Dynamic analysis of malware attack and its defence in computer networks. In: *The 1st international conference on mathematics and applied data science (ICMADS'25)* 29-31 August 2025, Konya, Türkiye, pp 117
54. Security.com. (2024) Wannacry ransomware attack: everything you need to know
55. Wikipedia contributors (2024) Wannacry ransomware attack. https://en.wikipedia.org/wiki/WannaCry_ransomware_attack
56. Vastare AR, KR N, Aiman U, et al (2025) SQL injection detection and prevention techniques using traditional and ai-based approaches. In: *2025 3rd international conference on data science and network security (ICDSNS)*, IEEE, pp 1–6
57. Casmiry EN, Sinde RS, Mduma NM (2026) A hybrid deep learning model for SQL injection attack detection. *IEEE Access* 14:6450–6463
58. OWASP (2024) SQL injection
59. OWASP (2024) SQL injection
60. Kinaneva DD, Hristov G V, Georgiev GD, Zahariev PZ (2025) Machine learning approaches for phishing detection using URL analysis. In: *2025 60th international scientific conference on information, communication and energy systems and technologies (ICEST)*, IEEE, pp 1–7
61. Wilk-Jakubowski JL, Pawlik L, Wilk-Jakubowski G, Sikora A (2025) Machine learning and neural networks for phishing detection: a systematic review (2017–2024). *Electronics* 14(18):374
62. Islam J, Patra C, Mani PK, Biswas S, Giri D, Maitra T (2025) Phishing url detection via machine learning: a comprehensive survey. In: *International conference on network security and blockchain technology*, Springer, pp 141–159
63. Gome V, Reis J, Alturas B (2020) Social engineering and the dangers of phishing. In: *2020 15th Iberian conference on information systems and technologies (CISTI)*, IEEE, pp 1–7
64. Zhraw DS, Hussain MA, Abduljabbar ZA, Nyangaresi VO, Aldarwish AJY (2025) Chronological review of MITM attacks: challenges, solutions and recommendations. In: *Computer science on-line conference*, Springer, pp 202–220
65. Negi D, Paruchuru V, Dokania A, Rallabandi M, Antara F, Jain A (2025) Optimizing cyber security measures against MITM attacks: a comparative analysis. In: *2025 International conference on engineering, technology and management (ICETM)*, IEEE, pp 1–6
66. Pritam Gajkumar Shah (2026) Arp poisoning and man-in-the-middle (MITM) attacks. *Australian Journal of Wireless Technologies Mobility and Security* 1(1)
67. Imperva (2024) What is a man-in-the-middle attack?
68. Li M, Dou Z (2023) Active eavesdropping detection: a novel physical layer security in wireless IoT. *EURASIP J Adv Signal Process* 1:119
69. Kapicak L, Stipal J, Trubak K, Machnik P, Si PL (2024) Detection system for passive network eavesdropping. In: *2024 15th IFIP wireless and mobile networking conference (WMNC)*, IEEE, pp 41–46
70. Huihui W, Zhang Y, Shen Y, Jiang X, Taleb T (2023) Achieving covertness and secrecy: the interplay between detection and eavesdropping attacks. *IEEE Internet Things J* 11(2):3233–3249
71. Farhan M, Abosata N (2023) Eavesdropping in bluetooth networks. *Int J Curr Eng Technol* 13(2):579–585
72. Proofpoint (2024) Eavesdropping | threat reference guide
73. PortSwigger. Cross-site scripting (xss), n.d. Accessed: 2025-06-19
74. Wajahat A, He J, Zhu N, Mahmood T, Nazir A, Ullah F, Qureshi S, Osman M (2024) An effective deep learning scheme for android malware detection leveraging performance metrics and computational resources. *Intell Decis Technol* 18(1):33–55
75. Wajahat A, He J, Zhu N, Mahmood T, Nazir A, Pathan MS, Qureshi S, Ullah F (2023) An adaptive semi-supervised deep learning-based framework for the detection of android malware. *J Intell Fuzzy Syst* 45(3):5141–515
76. Saied M, Guirguis S, Madbouly M (2024) Review of artificial intelligence for enhancing intrusion detection in the internet of things. *Eng Appl Artif Intell* 127:107231

77. Saied M, Guirguis S, Madbouly M (2023) A comparative study of using boosting-based machine learning algorithms for IoT network intrusion detection. *Int J Computat Intell Syst* 16(1):177
78. Mohamed Saied Essa and Shawkat Kamal Guirguis (2023) Evaluation of tree-based machine learning algorithms for network intrusion detection in the internet of things. *IT Professional* 25(5):45–56
79. Saied M, Guirguis S, Madbouly M (2023) A comparative analysis of using ensemble trees for botnet detection and classification in IoT. *Sci Rep* 13(1):21632
80. Ajayan A, Kirubavathi G, Sarker IH (2025) Distilxids: efficient, lightweight and explainable transformer-based language model for real-time network intrusion detection. *Neurocomputing* 132398
81. Zhong Y, Miao Q, Chen Y, Deng J, Cheng Y, Xu W (2025) Attention is all you need to defend against indirect prompt injection attacks in llms. *arXiv preprint [arXiv:2512.08417](https://arxiv.org/abs/2512.08417)*
82. Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S, Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler DM, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, Amodei D (2020) Language models are few-shot learners
83. Yupeng Chang X, Wang JW, Yuan W, Yang L, Zhu K, Chen H, Yi X, Wang C, Wang Y et al (2024) A survey on evaluation of large language models. *ACM Trans Intell Syst Technol* 15(3):1–45
84. Zhao WX, Zhou K, Li J, Tang T, Wang X, Hou Y, Min Y, Zhang B, Zhang J, Dong Z et al. (2023) A survey of large language models. *arXiv preprint [arXiv:2303.18223](https://arxiv.org/abs/2303.18223)*
85. Hadi MU, Qureshi R, Shah A, Irfan M, Zafar A, Shaikh MB, Akhtar N, Wu J, Mirjalili S, et al. (2023) A survey on large language models: applications, challenges, limitations, and practical usage. *Authorea Preprints*,
86. Zibaeirad A, Vieira M (2025) Reasoning with llms for zero-shot vulnerability detection. *arXiv preprint [arXiv:2503.17885](https://arxiv.org/abs/2503.17885)*,
87. Xu T, Wen Z, Zhao X, Wang J, Li Y, Liu C (2025) L2m-aid: Autonomous cyber-physical defense by fusing semantic reasoning of large language models with multi-agent reinforcement learning (preprint). *arXiv preprint [arXiv:2510.07363](https://arxiv.org/abs/2510.07363)*
88. Mahmood MAI, Ashab F, Sohan MS, Chy MSI, Kader MF (2025) LLM-enhanced security framework for IoT network: anomaly detection and malicious devices identification. *IEEE Access*
89. Bui M-T, Boffa M, Valentim RV, Navarro JM, Chen F, Bao X, Houidi ZB, Rossi D (2024) A systematic comparison of large language models performance for intrusion detection. *Proceed ACM Netw 2(CoNEXT4)*:1–23
90. Karunanayake B, Khalil I, Yi X, Lam K-Y (2025) Toward LLM-driven adaptive policy orchestration for host-based intrusion detection systems in IoT environments. *IEEE Network*
91. Singh Y, Patel ND, Shandilya SK (2024) Enhancing security operations center efficiency through multi-model integration of large language models and siem systems. *ResearchSquare Preprints*
92. Saha D, Tarek S, Yahyaie K, Saha SK, Zhou J, Tehranipoor M, Farahmandi F (2024) A paradigm shift. *IEEE Access*, LLM for soc security
93. Saheed YK, Chukwuere JE (2025) Autonomous LLM agent: a memory-augmented, edge-optimized shap explanations with zero-day attack resilience in IoT/industrial IoT networks. *IEEE Internet of Things Journal*
94. Fumero S, Huang K, Boffa M, Giordano D, Mellia M, Houidi ZB, Rossi D (2025) Cybersleuth: Autonomous blue-team LLM agent for web attack forensics. *arXiv preprint [arXiv:2508.20643](https://arxiv.org/abs/2508.20643)*
95. Li T, Zhu Q (2025) Agentic ai for cyber resilience: a new security paradigm and its system-theoretic foundations. *arXiv preprint [arXiv:2512.22883](https://arxiv.org/abs/2512.22883)*
96. Rigaki M, Lukáš O, Catania C, Garcia S (2024) Prompt. exploit. repeat: automating network security testing with LLMs. In: *International conference on agents and artificial intelligence*, Springer, pp 15–36
97. Nandi SK, Ratti R, Singh SR, Nandi S (2025) Prompt engineering-based network intrusion detection system. *IEEE Access* 13:190859–190871
98. Ahmed M, Wei J, Al-Shaer E (2024) Prompting LLM to enforce and validate cis critical security control. In: *Proceedings of the 29th ACM symposium on access control models and technologies*, pp 93–104,
99. Maria Rigaki, Carlos Catania, and Sebastian Garcia. Hackphyr: A local fine-tuned LLM agent for network security environments. *arXiv preprint [arXiv:2409.11276](https://arxiv.org/abs/2409.11276)*, 2024
100. Lodh S, Obaidat I, Rustam F, Jurcut AD (2025) Lightweight fine-tuning of LLMs for explainable intrusion detection in sdn. In: *2025 21th International conference on wireless and mobile computing, networking and communications (WiMob)*, IEEE, pp 1–6
101. Li X, Zheng Z, Zhao M, Zhao Y, Shi L, Wang B (2025) Rlfe-ids: A framework of intrusion detection system based on retrieval augmented generation and large language model. *Computer Networks*, 111341
102. Ikbarieh S, Aryal K, Gupta M (2025) Rag-targeted adversarial attack on LLM-based threat detection and mitigation framework. *arXiv preprint [arXiv:2511.06212](https://arxiv.org/abs/2511.06212)*

103. Gao X, Chang X (2025) A retrieval-augmented generation framework based on a knowledge graph of cybersecurity vulnerabilities in power networks. *IEEE Access* 13:186693–186710
104. Podpora M, Baranowski M, Chopcian M, Kwasniewicz L, Radziewicz W (2025) LLM firewall using validator agent for prevention against prompt injection attacks. *Appl Sci* 16(1):85
105. Neha Prakash Joshi (2024) Cybersecurity threat detection using multimodal LLMS. *Int J Res Appl Innov* 7(6):11636–11639
106. Luay M, Layeghy S, Pandey Y, Kulatilleke G, Portmann M (2025) Multimodal LLMS for zero-shot intrusion detection using netflow visualisations. In: 2025 IEEE 50th Conference on local computer networks (LCN), IEEE, pp 1–7
107. Masukawa R, Yun S, Jeong S, Huang W, Ni Y, Bryant I, Bastian ND, Imani M (2025) Packetclip: Multi-modal embedding of network traffic and language for cybersecurity reasoning. *arXiv preprint arXiv:2503.03747*
108. Qureshi S, Li J, Akhtar F, Tunio S, Khand ZH, Wajahat A (2021) Analysis of challenges in modern network forensic framework. *Secur Commun Netw* 1:8871230
109. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. *Advances in neural information processing systems*, 30
110. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020) Language models are few-shot learners. *Adv Neural Inf Process Syst* 33:1877–1901
111. Lewis P, Perez S, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih W, Rocktäschel T et al (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv Neural Inf Process Syst* 33:9459–9474
112. Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan KR, Cao Y (2022) React: synergizing reasoning and acting in language models. In: The 11th international conference on learning representations
113. Schick T, Dwivedi-Yu J, Dessi R, Raileanu R, Lomeli M, Hambro E, Zettlemoyer L, Cancedda N, Scialom T (2023) Toolformer: language models can teach themselves to use tools. *Adv Neural Inf Process Syst* 36:68539–68551
114. Mahmoodi M, Jameii SM (2024) Utilizing large language models for DDOS attack detection. In: 2024 OPJU International technology conference (OTCON) on smart computing for innovation and advancement in industry 4.0, IEEE, pp 1–6
115. Yin Z, Liu S, Xu G (2024) Drllm: Prompt-enhanced distributed denial-of-service resistance method with large language models. *arXiv preprint arXiv:2409.10561*
116. Ferrag MA, Ndhlovu M, Tihanyi N, Cordeiro LC, Debbah M, Lestable T, Thandi NS (2024) Revolutionizing cyber threat detection with large language models: a privacy-preserving Bert-based lightweight model for iot/iiot devices. *IEEE Access* 12:23733–23750
117. Chen S, Piet J, Sitawarin C (2024) and David Wagner. Defending against prompt injection with structured queries. *Struq*
118. Wang T, Xie X, Zhang L, Wang C, Zhang L, Cui Y (2024) Shieldgpt: an LLM-based framework for DDOS mitigation. In: Proceedings of the 8th Asia-Pacific Workshop on Networking, pp 108–114
119. Balasubramanian P, Ali T, Salmani M, KhoshKholgh D, Kostakos P (2024) Hex2sign: automatic ids signature generation from hexadecimal data using LLMS. In: 2024 IEEE International conference on big data (BigData), IEEE, pp 4524–4532
120. Yin Z, Liu S, Xu G (2025) Drllm: Prompt-enhanced distributed denial-of-service resistance method with large language models. In: ICASSP 2025–2025 IEEE international conference on acoustics, speech and signal processing (ICASSP), IEEE, pp 1–5
121. Siddiq ML, Zhang J, Roney L, Santos JCS (2024) Re (gexl dos) eval: evaluating generated regular expressions and their proneness to dos attacks. In Proceedings of the 2024 ACM/IEEE 44th international conference on software engineering: new ideas and emerging results, pp 52–56
122. Peng C, Wang H, Wu Y, Wu H, Fan M, Zhao Y, Liu T (2025) Etrace: Event-driven vulnerability detection in smart contracts via LLM-based trace analysis. *arXiv preprint arXiv:2506.15790*
123. Zhang Q, Xiong Z, Mao ZM (2024) Safeguard is a double-edged sword: Denial-of-service attack on large language models. *arXiv preprint arXiv:2410.02916*
124. Uddin MA, Sarker IH (2024) An explainable transformer-based model for phishing email detection: a large language model approach. *arXiv preprint arXiv:2402.13871*
125. Koide T, Fukushi N, Nakano H, Chiba D (2024) Chatspamdetector: leveraging large language models for effective phishing email detection. *arXiv preprint arXiv:2402.18093*
126. Lee J, Lim P, Hooi B, Divakaran DM (2024) Multimodal large language models for phishing webpage detection and identification. *arXiv preprint arXiv:2408.05941*
127. Koide T, Fukushi N, Nakano H, Chiba D (2023) Detecting phishing sites using chatgpt. *arXiv preprint arXiv:2306.05816*

128. Kulkarni A, Balachandran V, Divakaran DM, Das T (2025) From ml to LLM: evaluating the robustness of phishing web page detection models against adversarial attacks. *Digital Threats Res Pract* 6(2):1–25
129. Xue Y, Spero E, Koh YS, Russello G (2025) Multiphishguard: an LLM-based multi-agent system for phishing email detection. *arXiv preprint arXiv:2505.23803*
130. Thapa J, Chahal G, Gabreanu ŞV, Otoum Y (2025) Phishing detection in the gen-ai era: Quantized llms vs classical models. In: 2025 IEEE/ACIS 29th International conference on software engineering, artificial intelligence, networking and parallel/distributed computing (SNPD), IEEE, pp 856–863
131. Gurve MK, Nitin Y P, Cahay M (2025) A machine learning approach to detecting phishing attacks using LLM-based embeddings. In: International conference on emerging trends and technologies on intelligent systems, Springer, pp 79–91
132. Alasmari SM, Sakly H, Kraiem N, Algarni A (2025) Phishing detection in IoT: an integrated CNN-ISTM framework with explainable AI and LLM-enhanced analysis. *Discover Internet of Things* 5(1):102
133. Weinz M, Zannone N, Allodi L, Apruzzese G (2025) The impact of emerging phishing threats: Assessing Quishing and LLM-generated phishing emails against organizations. In: Proceedings of the 20th ACM Asia conference on computer and communications security, pp 1550–1566
134. Park H, Lee J, Han S, Byun H (2025) Enhanced voice phishing detection using an LLM-based framework for data augmentation and classification. *IEEE Access*
135. Fairbanks J, Serra E (2024) Generating phishing attacks and novel detection algorithms in the era of large language models. In: 2024 IEEE international conference on big data (BigData), IEEE, pp 2314–2319
136. Afane K, Wei W, Mao Y, Farooq J, Chen J (2024) Next-generation phishing: how LLM agents empower cyber attackers. In: 2024 IEEE International Conference on Big Data (BigData), IEEE, pp 2558–2567
137. Hafzullah İŞ (2024) LLM-driven sat impact on phishing defense: a cross-sectional analysis. In: 2024 12th International symposium on digital forensics and security (ISDFS), IEEE, pp 1–5
138. Chen F, Wu T, Nguyen V, Rudolph C (2025) SOK: Exposing the generation and detection gaps in LLM-generated phishing through examination of generation methods, content characteristics, and countermeasures. *arXiv preprint arXiv:2508.21457*
139. Pang Y, Meng W, Liao X, Wang T (2025) Paladin: Defending LLM-enabled phishing emails with a new trigger-tag paradigm. *arXiv preprint arXiv:2509.07287*
140. Abuadba A, Wang S, Almashor M, Ahmed ME, Gaire R, Camtepe S, Nepal S (2022) Towards web phishing detection limitations and mitigation. *arXiv preprint arXiv:2204.00985*
141. Moura GC, Daniels T, Bosteels M, Castro S, Müller M, Wabeke T, van den Hout T, Korczyński M, Smaragdakis G (2024) Characterizing and mitigating phishing attacks at cctld scale. In: Proceedings of the 2024 on ACM SIGSAC conference on computer and communications security, pp 2147–2161
142. Nakano H, Koide T, Chiba D (2025) Phishparrot: LLM-driven adaptive crawling to unveil cloaked phishing sites. *arXiv preprint arXiv:2508.02035*
143. Lin M, Zhang H, Lao J, Li R, Zhou Y, Yang C, Cao Y, Tang M (2025) Are your LLM-based text-to-SQL models secure? exploring SQL injection via backdoor attacks. *Proc ACM Manage Data* 3(6):1–27
144. Tasdemir K, Khan R, Siddiqui F, Sezer S, Kurugollu F, Yengce-Tasdemir SB, Bolat A (2023) Advancing SQL injection detection for high-speed data centers: a novel approach using cascaded NLP. *arXiv preprint arXiv:2312.13041*
145. Zhang Y, Jiang Z, Cheng X, Wu H, Tian Z, Lou S (2024) A method of SQL injection attack detection based on large language models. In: 2024 2nd International conference on computer network technology and electronic and information engineering (CNTEIE), IEEE, pp 154–158
146. Yang T, Jiang Z, Wang Y (2024) Llmsqli: A black-box web SQLi detection tool based on large language model. In: 2024 5th International conference on big data and artificial intelligence and software engineering (ICBASE), IEEE, pp 629–633
147. Gui Z, Wang E, Deng B, Zhang M, Chen Y, Wei S, Xie W, Wang B (2024) Sqlipt: Evaluating and utilizing large language models for automated SQL injection black-box detection. *Appl Sci* 14(16):6929
148. Junze H, Jin X, Zeng Y, Liu Y, Li Y, Dan D, Xie K, H Zhu (2025) Automated code vulnerability discovery via LLM and static code analysis integration, Qlpro
149. Leung D, Tsai O, Hashemi K, Tayebi B, Tayebi MA (2024) Xploitsql: Advancing adversarial SQL injection attack generation with language models and reinforcement learning. In: Proceedings of the 33rd ACM International conference on information and knowledge management, pp 4653–4660
150. Ali I, Adil SH, Ebrahim M (2020) Intrusion detection framework for SQL injection
151. Pedro R, Castro D, Carreira P, Santos N (2023) From prompt injections to SQL injection attacks: how protected is your LLM-integrated web application? *arXiv preprint arXiv:2308.01990*
152. Pedro R, Coimbra ME, Castro D, Carreira P, Santos N (2025) Prompt-to-SQL injections in LLM-integrated web applications: risks and defenses. In: Proceedings of the IEEE/ACM 47th International conference on software engineering, pp 1768–1780

153. Maligazhdarova N, Avinash BM, Mukasheva A, Yedilkhan D, Askhatuly A, Berdyshev A (2025) A comparative study of machine learning and large language models for sql and nosql injection vulnerability detection. In: 2025 IEEE 5th international conference on smart information systems and technologies (SIST), IEEE, pp 1–7
154. Tete SB (2024) Threat modelling and risk analysis for large language model (LLM)-powered applications. arXiv preprint [arXiv:2406.11007](https://arxiv.org/abs/2406.11007)
155. Babaei V, Ravindran A (2024) Gensqli: A generative artificial intelligence framework for automatically securing web application firewalls against structured query language injection attacks. *Fut Internet* 17(1):8
156. Lin M, Zhang H, Lao J, Li R, Zhou Y, Yang C, Cao Y, Tang M (2025) Toxicsql: Migrating SQL injection threats into text-to-SQL models via backdoor attack. arXiv preprint [arXiv:2503.05445](https://arxiv.org/abs/2503.05445)
157. Mui R, Frankl P (2010) Preventing sql injection through automatic query sanitization with assist. arXiv preprint [arXiv:1009.3712](https://arxiv.org/abs/1009.3712)
158. Jahanshahi R, Doupé A, Egele M (2020) You shall not pass: mitigating SQL injection attacks on legacy web applications. In: Proceedings of the 15th ACM Asia conference on computer and communications security, pp 445–457
159. Kim S, Lee S (2025) Research on SQL injection fuzzing using LLM. *J Korea Inst Inf Secur Cryptol* 35(2):323–334
160. Dat NLQ, Anh NLQ, Thang NM (2025) AI-enhanced SQL injection detection framework: a novel approach combines LLMS with traditional fuzzing to improve web application vulnerability detection. *Journal of Science and Technology on Information security* 70–78
161. Jamdade M, Liu Y (2024) A pilot study on secure code generation with chatgpt for web applications. In: Proceedings of the 2024 ACM Southeast conference, pp 229–234
162. Zhao J, Chen K, Zhang W, Yu N (2024) SQL injection jailbreak: a structural disaster of large language models. arXiv preprint [arXiv:2411.01565](https://arxiv.org/abs/2411.01565)
163. Shekhar K, Sushma V, Umesh B, Deepika K, Ashish Y (2025) Hybrid text mining and generative-AI framework for SQL injection attack detection. *Int J Commun Netw Inf Secur* 17(3):744–755
164. Zahan N, Burckhardt P, Lysenko M, Aboukhadijeh F, Williams L (2024) Leveraging large language models to detect NPM malicious packages
165. Sánchez Sánchez PM, Celdrán A H, Bovet G, Pérez GM (2024) Transfer learning in pre-trained large language models for malware detection based on system calls. In: MILCOM 2024–2024 IEEE Military communications conference (MILCOM), pp 853–858. IEEE
166. Qian X, Zheng X, He Y, Yang S, Cavallaro L (2025) Lamd: Context-driven android malware detection and classification with LLMS. In: 2025 IEEE Security and privacy workshops (SPW), IEEE, pp 126–136
167. Omar M, Zangana HM, Al-Karaki J N, Mohammed D (2024) Harnessing LLMS for IoT malware detection: a comparative analysis of Bert and GPT-2. In: 2024 8th International symposium on multi-disciplinary studies and innovative technologies (ISMSIT), IEEE, pp 1–6
168. Devadiga D, Jin G, Potdar B, Koo H, Han A, Shringi A, Singh A, Chaudhari K, Kumar S (2023) Glean: Gan and LLM for evasive adversarial malware. In: 2023 14th International conference on information and communication technology convergence (ICTC), IEEE, pp 53–58
169. Zhao W, Juntao W, Meng Z (2025) Apppoet: Large language model based android malware detection via multi-view prompt engineering. *Expert Syst Appl* 262:125546
170. Dong M, Liu L, Guo Q, Bai H, Gong R, Bai Y, He W, Wang Z, Xu G, Zhang J (2025) L eodroid: an LLM-based few-shot multi-label detection for android malware. In: 2025 IEEE security and privacy workshops (SPW), IEEE, pp 294–306
171. Shafee S, Bessani A, Ferreira PM (2025) False alarms, real damage: adversarial attacks using LLM-based models on text-based cyber threat intelligence systems. arXiv preprint [arXiv:2507.06252](https://arxiv.org/abs/2507.06252)
172. Lan T, Nait-Abdesslem F (2025) LLM-driven feature-level adversarial attacks on android malware detectors. arXiv preprint [arXiv:2512.21404](https://arxiv.org/abs/2512.21404)
173. R Sun, M Botacin, N Sapountzis, X Yuan, M Bishop, DE Porter, X Li, A Gregio, D Oliveira (2020) Leveraging uncertainty for effective malware mitigation, a praise for defensive programming
174. von der Assen J, Celdrán A H, Sefa R, Bovet G, Stiller B (2023) MTFS: a moving target defense-enabled file system for malware mitigation
175. Guo W, Xue J, Liu Z, Han W, Hu J (2025) Malgta: large language model-based guided malware tactical analysis: W. guo et al. *The Journal of Supercomputing*, 81(9):1049
176. Kshetri N, Kumar D, Hutson J, Kaur N, Osama OF (2024) algoxssf: Detection and analysis of cross-site request forgery (xsrif) and cross-site scripting (xss) attacks via machine learning algorithms. In: 2024 12th international symposium on digital forensics and security (ISDFS), IEEE, pp 1–8
177. Steinhauser A, Tuuma P (2019) Djangochecker: Applying extended taint tracking and server side parsing for detection of context-sensitive xss flaws. *Software: Practice and Experience* 49(1):130–148

178. Miczek D, Gabbireddy D, Saha S (2025) Leveraging LLM to strengthen ml-based cross-site scripting detection. arXiv preprint [arXiv:2504.21045](https://arxiv.org/abs/2504.21045)
179. Adiputra A A, Hwang Y, Son J, Kim H, et al (2025) Fine-tuning transformer llms for detecting SQL injection and xss vulnerabilities. In: 2025 International conference on artificial intelligence in information and communication (ICAIIIC), IEEE, pp 0946–0951
180. Tanaka Y, Mimura M (2025) Detection of cross-site scripting attacks using large language models. Available at SSRN 5218916
181. Pazos J C, Legare J-S, Beschastnikh I, Aiello W (2020) Precise xss detection and mitigation with client-side templates
182. Kirda E, Kruegel C, Vigna G, Jovanovic N (2006) Noxes: a client-side solution for mitigating cross-site scripting attacks. In: Proceedings of the 2006 ACM symposium on Applied computing, pp 330–337
183. Zhou Y, Wang E, Yang W, Ge W, Yang S, Zhang Y, Wei Q, Xie W (2025) Xsshield: Defending against stored xss attacks using LLM-based semantic understanding. Appl Sci 15(6):3348
184. Babaey V, Ravindran A (2025) Genxss: an AI-driven framework for automated detection of xss attacks in wafs. In SoutheastCon 2025, IEEE, pp 1519–1524
185. Lee C, Sohn I, Lee W (2022) Eavesdropping detection in bb84 quantum key distribution protocols. IEEE Trans Netw Serv Manage 19(3):2689–2701
186. Yao X, Huang K, Chen Y, Guo J, Tang J, Zhao M (2025) {EchoLLM}:{LLM-Augmented} acoustic eavesdropping attack on bone conduction headphones with {mmWave} radar. In: 34th USENIX security symposium (USENIX Security 25), pp 6619–6638
187. Cui J, Li Z, Xing L, Liao X (2025) Safeguard-by-development: a privacy-enhanced development paradigm for multi-agent collaboration systems. arXiv preprint [arXiv:2505.04799](https://arxiv.org/abs/2505.04799)
188. Fellan A, Djuitcheu H, Lindenschmitt D, Schotten HD (2024) Exploring jamming threats and defense mechanisms in future 6g communication systems. In: MILCOM 2024-2024 IEEE Military communications conference (MILCOM), IEEE, pp 117–122
189. Nagarajan S, Kamalbabu K (2024) Analysis of how ChatGPT and Gemini help in the generation of cyber attacks. In: 2024 8th International conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)(I-SMAC), IEEE, pp 663–667
190. Zhang C, Sun G, Wang J, Lin Y, Yuan W, Coleri S, Niyato D, Quek T Q (2025) Large AI model-enabled secure communications in low-altitude wireless networks: concepts, perspectives and case study. arXiv preprint [arXiv:2508.00256](https://arxiv.org/abs/2508.00256)
191. Jin W, Cao Y, Su J, Wang D, Zhang Y, Xue M, Hao J, Dong JS, Yang Y (2025) Whispering under the eaves: protecting user privacy against commercial and LLM-powered automatic speech recognition systems. arXiv preprint [arXiv:2504.00858](https://arxiv.org/abs/2504.00858)
192. Zhang Y, Ouyang W, Zhang Y, Cheng L, Wu C, Hu W (2025) Automatic generation of a cryptography misuse taxonomy using large language models
193. TejeroFernández JJ, Sánchez-Macián A (2025) Evaluating language models for threat detection in iot security logs
194. Yan B, Zhou Z, Zhang X, Li C, Zeng R, Qi Y, Wang T, Zhang L (2025) Attack the messages, not the agents: a multi-round adaptive stealthy tampering framework for LLM-mas. arXiv preprint [arXiv:2508.03125](https://arxiv.org/abs/2508.03125)
195. Mo K, Hu L, Long Y, Li Z (2025) Attractive metadata attack: Inducing LLM agents to invoke malicious tools. arXiv preprint arxiv.org/abs/2508.02110
196. Wen H, Sharma P, Yegneswaran V, Gehani A, Porras P, Lin Z (2025) Mobillm: An agentic ai framework for closed-loop threat mitigation in 6g open rans. In: MILCOM 2025-2025 IEEE Military communications conference (MILCOM), IEEE, pp 1–6
197. ElZemity A, Arief B, Li S (2025) Cyberllminstruct: A new dataset for analysing safety of fine-tuned LLMS using cyber security data. arXiv preprint [arXiv:2503.09334](https://arxiv.org/abs/2503.09334)
198. Huang L, Dave D, Jin M, Cody T, Beling P (2025) From capabilities to performance: evaluating key functional properties of llm architectures in penetration testing. arXiv preprint [arXiv:2509.14289](https://arxiv.org/abs/2509.14289)
199. Shaikh A, Varol A (2025) and Johanna Virkki. Man-in-the-middle attacks on LLM-enabled vacuum robots. IEEE Access, From prompts to motors
200. Zhang Y, Gao Y, Hu W, Li G, Li J (2025) Openeyes: Leveraging LLM and graph neural networks for deep ransomware detection. In: 2025 IEEE Conference on cloud and big data computing (CBDCOM), IEEE, pp 181–188
201. Wajahat A, He J, Zhu N, Mahmood T, Saba T, Khan A, Alamri F (2024) Outsmarting android malware with cutting-edge feature engineering and machine learning techniques. Comput Mater Continua 79(1):651
202. Nazir A, Wajahat A, Akhtar F, Ullah F, Qureshi S, Malik SA, Shakeel A (2020) Evaluating energy efficiency of buildings using artificial neural networks and k-means clustering techniques. In: 2020 3rd

- International conference on computing, mathematics and engineering technologies (iCoMET), IEEE, pp 1–7
203. OWASP Top OWASP. for large language model applications, owasp foundation. <https://owasp.org/www-project-top-10-for-large-language-model-applications>, 10
 204. Chen Y, Li H, Zheng Z, Wu D, Song Y, Hooi B (2025) Defense against prompt injection attack by leveraging attack techniques. In: Proceedings of the 63rd Annual meeting of the association for computational linguistics vol 1, Long Papers, pp 18331–18347
 205. Sarker IH (2024) Llm potentiality and awareness: a position paper from the perspective of trustworthy and responsible AI modeling. *Discov Artif Intell* 4(1):40
 206. Wajahat A, Zhang K, Latif J, Ma X, Ahad A, Ahmed KI (2026) Dynamic weighted federated learning: a scalable and privacy-centric approach to android IoT malware detection. *Concurr Computat Pract Exper* 38(1):e70441
 207. Azeem MZ, Imran A, Akhtar F, Wajahat A, Latif J, Memon SA (2020) Effects of code cloning in mobile applications. In: 2020 3rd International conference on computing, mathematics and engineering technologies (iCoMET), IEEE, pp 1–6
 208. Saifullah K, Khan MI, Jamal S, Sarker IH (2024) Cyberbullying text identification: a deep learning and transformer-based language modeling approach. In: EAI endorsed transactions on industrial networks and intelligent systems
 209. Prasad A, Yadav V, Solanki C, Goswami H, Jha T, Nagal D (2025) Stealthphisher: A defensive framework against phishing attack using hybrid deep learning and Genai. *Expert Syst Appl* 130205
 210. Prasad A, Chandra S, Uddin M, Al-Shehari T, Alsadhan NA, Ullah SS (2024) Permgaurd: A scalable framework for android malware detection using permission-to-exploitation mapping. *IEEE Access*
 211. Uddin MA, Islam MN, Maglaras L, Janicke H, Sarker IH (2025) Explainabledetector: Exploring transformer-based language modeling approach for SMS spam detection with explainability analysis. *Digital Communications and Networks*

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Lamia Chaari Fourati is a full professor at the Computer Science and Multimedia Higher Institute (ISIMS), Sfax University, Tunisia, and a senior researcher at the Digital Research Center of Sfax (CRNS), within the Laboratory of Signals, systems, aRtificial Intelligence, neTworkS (SM@RTS). She leads the INTELLECT research team, focusing on intelligent networks and telecommunication protocols for emerging communication technologies. Her research is dedicated to the design, analysis, and validation of novel protocols and intelligent mechanisms for next-generation digital telecommunication networks. Her expertise spans wireless access networks, wireless sensor networks, vehicular networks (VANETs and IoV), the Internet of Things (IoT), software-defined networking (SDN), information-centric networking (ICN), wireless body area networks (WBAN), and aerial networks involving unmanned aerial vehicles (UAVs) and flying ad hoc

networks (FANETs). Prof. Chaari Fourati's work addresses key challenges related to quality of service (QoS) provisioning, resource management, cybersecurity, trust, privacy, and resilience in network systems. Her applied research also contributes to societal well-being, with a strong focus on sustainable and secure digital technologies for healthcare systems, water and air quality monitoring, and intelligent transportation. This includes the development of smart, energy-efficient, and secure communication frameworks for critical systems like remote health monitoring and environmental control. She is the author of over 140 journal publications and more than 160 conference papers. Her scientific contributions have been widely recognized by the academic and engineering communities.



Mirna Awad is a researcher and lecturer at École de technologie supérieure (ÉTS), Montréal, Canada. She holds a master's degree in business computer science from Lebanese University, Lebanon. Her work focuses on cloud computing, NFV, and intelligent resource management in distributed systems. She has contributed to several publications on system optimization and cyberattack detection in next-generation networks.



Mariem Ben Ali received her master's degree in computer science in 2025 from the Higher Institute of Computer Science and Multimedia of Sfax (ISIMS), University of Sfax, Tunisia. She conducted her master's research project at the École de technologie supérieure (ÉTS), Montréal, Canada, where she contributed to advanced research activities in intelligent and secure systems. Her research interests focus on cybersecurity, large language models (LLMs), and network resilience, with particular emphasis on the development of robust and adaptive defense mechanisms for modern networked environments. Her work explores the integration of AI-driven techniques for cyberattack detection, mitigation, and system reliability, targeting next-generation infrastructures and distributed systems.



Wael Jaafar is an associate professor in the Department of Software and IT Engineering at the École de Technologie Supérieure (ETS), University of Quebec, Montreal, Canada, since September 2022. He received his Master's and Ph.D. degrees from Polytechnique Montreal, Canada. From 2019 to 2022, he was a postdoctoral fellow in the Department of Systems and Computer Engineering at Carleton University, supported by an NSERC Postdoctoral Fellowship. Prior to that, he spent four years (2014–2018) working in the telecommunications industry. Dr. Jaafar has held visiting researcher positions at Khalifa University (UAE, 2019), Keio University (Japan, 2013), and the Université du Québec à Montréal (Canada, 2007). He serves as an editor for IEEE Communications Letters, SciFinity Computing & AI Connect, and Elsevier Physical Communication. He is the recipient of several distinctions, including Best Paper Awards at IEEE ICC 2021, IEEE ISCC 2023, and CIoT 2024, the NSERC Alexander Gra-

ham Bell Graduate Scholarship, and an FRQNT international internship scholarship. His current research interests include wireless communications, integrated terrestrial and non-terrestrial networks, resource allocation, machine learning for communication systems and networks, cybersecurity, digital twins, quantum computing, and quantum AI.